

owing to the inadequacy of a context-free grammar to fully define the language. Some examples of semantic tests required in Algol are:

- Consistency of an identifier's attributes with its appearance in the source. A statement label should not be used as an expression variable, a simple variable should not appear with an index, a non-typed procedure should not appear as a function, etc.
- Matched appearances of identifiers. In a procedure declaration, the formal parameters are first listed, then declared. Each declaration must correspond to one of the names in the list and vice versa.
- Limitations imposed by an implementation on the sizes of arrays, or the number of variables. These depend on the magnitude of various constants.
- Overflow in numeric arithmetic performed by the compiler.

Semantic errors are very easy to diagnose accurately, since the nature of the problem is apparent from the apply action. Recovery requires some attention to the possible side-effects. In general, some simple action should be chosen that removes the semantic problem with no undesirable side-effects.

An undefined identifier should be reported and then defined by entering it in the symbol table. Although its attributes are not always clear from its context, there is usually sufficient information that some broad default attributes can be chosen. If necessary, that symbol table entry can simply be marked "entered through error." Then any subsequent misuse of the identifier will not be reported.

Identifier misuse occurs through the appearance of some declared identifier in a context that requires a different kind of declaration. For example, a statement label in Algol 60 cannot be used as a data item; it does not stand for a numeric value. When a misuse is detected, it is usually because of a multiple use of the same identifier. It is best, we feel, to add the attribute "error" to the identifier and to suppress further complaints centered on it.

A literal value found to be out of range can easily be adjusted to a default compatible with its context.

The possibility of overflow during compilation must be considered in any arithmetic operation involving source data. In order to frame a suitable diagnostic message and to recover gracefully, each such operation in the compiler must be identified and suitably protected. Overflow can occur through the conversion of number representations to internal values, through constant folding, or through memory allocation that exceeds the bounds of the internal integer representations. These occurrences clearly call for different kinds of diagnostic messages and recovery strategies.

12.3. Syntax Errors

A *syntax error* is detected by the parser upon encountering some token that is incompatible with its current state and stack. We shall discuss error recovery for an LR parser in some detail, although an error recovery strategy can be developed for any of the deterministic parsers discussed in chapters 4, 5, and 6.

Most error recovery systems assume that one of three kinds of error exist: insertion, deletion, or substitution. An *insertion* error results from the presence of a token in the source string that, if removed, would result in a valid sentence. An insertion error is defined by the following valid and invalid derivations:

$$\sim(S \rightarrow^+ xty) \text{ but } (S \rightarrow^+ xy)$$

Here t represents the incorrectly inserted token. A *deletion* error results from the inadvertent deletion of a token that is needed for syntactic validity:

$$\sim(S \rightarrow^+ xy) \text{ but } \dagger S \rightarrow^+ xty$$

Here t is needed to transform xy into a sentence. A *substitution* error is such that a change of one token into another will yield a valid sentence:

$$\sim(S \rightarrow^+ xty) \text{ but } (S \rightarrow^+ xuy)$$

Here a change of token t to u yields a sentence xuy .

If an error is one of these three kinds, we say that the error is *simple*. If, in addition, each pair of errors is separated by a fairly large number of tokens, we say that the errors are *sparse*. Most errors found in practice are simple and sparse.

Even if errors are sparse and simple, a parser cannot always determine which of the three kinds of error produced the error symptom. A substitution or insertion error usually results in an error symptom at the error token; however, the error symptom may also appear later. For example, a `BEGIN` inserted between two statements would be interpreted as a new block opening (assuming that a semicolon may be either a separator or a terminator of statements). The error symptom of the inserted `BEGIN` will not appear until much later in the program, when the absent `END` manifests itself. A deleted token can at best result in an error symptom on the token following the deletion; again, it is possible that several additional tokens are scanned before the error symptom is seen.

We see that an attempt to uncover the “real” error is usually impossible. At best, given full knowledge of the remaining input list, an error recovery strategy can indicate the least number of repairs to the source text that suffice to bring the source into syntax.

12.3.1. General Methods

Suppose that a program is known to have at most one error somewhere, of one of the three kinds above. Then the error can be identified in $O(n^2)$ operations, where n is the number of tokens in the program. We simply try each of the three error types in every possible token position, and parse the resulting string. Given one out of n sentence tokens, there is one insertion error, m replacement errors, and m deletion errors possible, where the language contains m terminal tokens. An LR parse requires Kn operations, for some fixed K , hence a single error repair (but not necessarily “the” error repair) can be identified in $K(1 + m^2)n^2$ operations.

If an unknown, but bounded, number of errors is known to exist, then the previous strategy can be repeated with combinations of 2, 3, 4, ..., k errors, where k is the upper bound on the number of errors present. A minimal repair set can then be identified in $K((1 + m^2)n^2)^k$ operations, or roughly $O(n^{2k})$ operations.

A strategy for unknown k might then be as follows. Let $k = 1$, and explore single errors. If a single error repair is found that yields a sentence, then halt. Otherwise, set $k := k + 1$, and search for k error repairs that yield a sentence. Continue to increment k and search for a k -error repair on each failure. A valid sentence must be found before $k = n$; however, if it must continue that far, we have performed $O(n^{2n})$ operations.

We clearly need a more efficient means of repairing errors. The bound $O(n^{2n})$ is much too large for any program of practical size.

Aho and Peterson (Aho [1972c]) describe an algorithm that will parse any input string to completion in a time proportional to the cube of the input length, provided the string contains only insertion, replacement, or deletion errors. Essentially, they show how error productions can be added to a grammar G to yield a grammar G' that accepts any string in G 's alphabet. A sentence is then parsed through Earley's algorithm (Earley [1970]); this algorithm will parse sentences in any context-free grammar in $O(n^3)$ operations. The parse may then be interpreted such that a minimum number of error productions are used in the parse.

Aho and Peterson have therefore shown that minimal error recovery can be achieved in $O(n^3)$ operations. For large n , this is still much too large for practical purposes. Their approach also requires a separate error parser, the Earley parser. The Earley parser may also be used for the error-free parse, but it is less efficient than an LR(1) parser in time and space.

12.3.2. Diagnosis of a Syntax Error

How meaningful can a syntax error diagnosis be to a programmer? Consider an LR(1) parser at an error symptom. The stack contains information about the previous history of the program, and the top-of-stack

state is a read state that expects a certain set of terminal tokens next; the error token is not among them.

The easiest diagnostic message to generate is a list of those tokens that are next expected; if the error token were one of them, then the error would not have occurred. For example, the message

```
ERROR IN LINE 19,000, COLUMN 13
A := WAS FOUND, BUT POSSIBILI-
TIES ARE: + - * / ↑ . . .
```

is a reasonable syntax error message. The list of possibilities is easily generated from the outsymbols of the read state in which the error was detected.

The top-of-stack read state is of course associated with some configuration of items as a result of the LR(1) parser construction (see chapter 6). It is conceivable that the configurations could be studied with the objective of generating an error message catalog tailored to the kinds of productions being worked on. Unfortunately, this can be an extremely laborious task—a typical grammar may yield over a thousand read states, each of which would have to be manually studied by someone. Any change in the grammar would render parts of the catalog useless, or at least require a reexamination of the messages and configurations.

Another diagnostic strategy is to simply report what was done to correct the error. The programmer is then often impressed by what the compiler is attempting to do for him and may well overlook the fact that the compiler is unable to provide a meaningful correction to the error symptom. If several tokens must be dropped or some tokens must be inserted, such a report would be useful to most users.

12.3.3. Patching a Syntax Error

At the point of an error symptom in a bottom-up LR parser, we have a stack of states. . . , S_2 , S_1 , S_0 , with S_0 on top, and a remaining input string $xa_1a_2 \dots$, where x is called the *error token*. The error token x and the top-of-stack state S_0 are such that S_0 is a READ or LOOKAHEAD state, and x is incompatible with that state. State S_0 can only be a READ state if the LOOKAHEAD tables are incomplete; see chapter 6 for details.

The error recovery system should arrive at a new state stack and input string such that parsing may continue. The new state stack need not be related to the old one at all; however, the new input string should be derived from the old one by only changing the first n tokens, with n reasonably small. We call such a modification to the state stack and input string a *patch*. A patch therefore consists of some combination of the following operations:

1. Deletion of one or more left-most tokens of the input string.
2. Insertion of one or more tokens as prefixes of the input string.
3. Removal of states in the stack.
4. Pushing one or more states onto the stack.

Now the state stack can only be modified into some stack that represents a viable prefix in the language; failure to ensure this condition will result in a catastrophic failure later in the parse. The easiest way of ensuring it is to only pop states from the stack or to add a state to the stack top that is compatible with the existing stack top. A new state Q to be pushed on the stack must be such that a transition from the top-of-stack state P to Q exists in the complete LR finite-state control.

The state stack represents a condensed history of all of the source material that has previously been parsed. It carries such information as the nesting in a parenthesized structure, whether one or more procedures have been entered, whether a declaration or an executable statement is being developed, etc. We contend that such information is useful in error recovery. For example, if the stack indicates that four BEGIN's have been seen so far, the chances are good that the four matching END's will appear past an error symptom.

Scanner Feedback

As much as possible of the remaining input string should be retained in order to provide some measure of syntax checking of the rest of the program. That is the whole point of error recovery. If the recovery strategy is to reject the rest of the string and to replace it with a string compatible with the state stack, then we really don't need error recovery.

Now an LR(1) parser will require the scanning of at most one token past a handle. A reasonable strategy that we shall develop in the next section is to make use of this next token and the stack contents to devise a patch. As an improvement, we may read several tokens past the error token, in an attempt to arrive at a more effective patch. However, the use of more than one token can introduce some difficulties if the scanner is dependent in any way upon the result of any parser apply actions.

For example, the scanner may be expected to classify identifiers by some criterion in order to return one of two or more token codes associated with identifiers. The scanner might have several states that guide its lexical decisions, the states being partially set by apply operations. If such is the case, we say that *feedback* exists between the apply operations and the scanner. It is very desirable that no feedback from the apply actions to the scanner exist.

If feedback exists, then the scanning of tokens past the error token is not reliable. There will be no apply operations resulting from such a scan to guide the scanning, and the token codes supplied by the scanner are likely to be incorrect.

Feedback should be avoided in the design of a compiler, regardless of the error recovery strategy or of the apparent need for it in the language. If necessary, the scanner may be based on some automaton whose state transitions are determined by the tokens scanned, in order to identify certain special tokens without feedback.

Given a feedback-free scanner, it is relatively easy to scan several tokens in the input string past the error token. These tokens may then be used either to construct a patch that is compatible with these tokens or to discard more input tokens until a patch can be devised.

Terminal or State Insertion

Consider a choice between inserting a terminal token T or pushing a state P . State P corresponds to some terminal or nonterminal token T' . If T' is terminal, then pushing P is clearly equivalent to inserting T' . Suppose that T' is nonterminal, and that pushing state P causes the parser to accept the next input token, and of course that P is compatible with the stack-top state. We then effectively have a sentential form

$$wST'xy$$

where wS represents the stack, with S the stack-top symbol, T' is the inserted nonterminal, x the error token, and y the remaining input list. Now the next derivation step must be of the form

$$wST'xy \Rightarrow wSt'xy$$

where a production $T' \rightarrow t'$ exists. If $|t'| = 1$, we again see that an insertion of t' is equivalent to pushing P ; by inserting t' , the LR(1) parser will reduce t' to T' , since t' must be the handle of $wSt'xy$.

We conclude from this discussion that if a nonterminal T' can derive a single terminal or nonterminal symbol, then any state whose insymbol is T' need never be pushed on the stack during error recovery; it is sufficient to insert one of the derived terminals. We need only consider pushing those states P whose insymbol T' is such that T' derives either the empty string or strings of length greater than 1.

A similar result holds for the insertion of terminal strings of length 1, with $l > 1$. However, the number of possible string insertions grows exponentially with the string length, and we feel that it is infeasible to consider the insertion of more than one terminal token.

Now most terminal token data structures are empty, while many of the nonterminal data structures are nonempty. Further, a simple default data structure can always be selected for those terminals with a structure, while the nonterminal data structures are often more complicated, perhaps requiring links to the symbol table or to other structures. It is clearly desirable to reduce the number of inserted nonterminals to a minimum. An inspection of the

grammar is sufficient to find those nonterminals for which no terminal insertion is an equivalent substitute. It is then feasible to add special error productions for each of these nonterminals, of the form $T \rightarrow x$, where $|x| = 1$ and x can be any terminal. The appearance of such a production can only occur during error recovery, but can be exploited by the apply actions to create a suitable data structure for T .

Given a grammar such that for every nonterminal T , a production of the form $T \rightarrow x$ exists, with $|x| = 1$, then nonterminal insertion as an error recovery strategy is unnecessary. Only terminal insertion, state dropping, or input token deletion is needed.

12.3.4. Semantics Operations in Error Recovery

Recall that an abstract-syntax tree (AST) is built as a linked list in a semantics stack, and that the semantics stack is associated with the parser's state stack (see chapter 7.) Now assume that some semantics operations are to be continued through a syntax error recovery. It is obviously vital that the integrity of the semantics stack structures be preserved through any error recovery patch.

If states can be removed in a patch, then no forward links in the state stack data structures should exist. That is, there should never be a pointer from some structure in the stack to a structure closer to the stack top. All pointers, if any, must be directed from the stack top inward.

If a state is pushed on the stack through a patch, then a data structure compatible with that state should be pushed as well. The structure will correspond in general to some terminal or nonterminal symbol in the grammar associated with the pushed state. If several kinds of structure are possible, the least complicated should be chosen. If necessary, special "null" structures might be devised in order to avoid difficulties in creating a suitable data structure.

We see that every new pushed state should be associated with some data structure valid for that state, and that pointers in a data structure should never point to structures associated with states higher in the stack. If these rules are rigidly adhered to, then the synthesis operations may be continued through an error patch to whatever level seems desirable for effective semantic error reporting.

12.3.5. A Bounded-Range Error Recovery Strategy

We first develop an error recovery system that does not require a lookahead of more than one token in the input string. It is therefore suitable for a compiler with feedback to the scanner.

1. *Upon encountering a syntax error*, the stack will consist of the states

$$S_n, S_{n-1}, \dots, S_1, S_0$$

where S_0 is the stack top state. Let X be the error token (next in the input list) and let S_0 be a READ state. Then X is not among the tokens in the transitions from S_0 . Set a flag ERFLAG that indicates that error recovery is in progress. Make a copy of the stack for the sake of stack restoration. Set variable $j := 0$; j will mark the current stack top. In the following steps, the operation “restore the stack” will result in the stack

$$S_n, S_{n-1}, \dots, S_{j+1}, S_j$$

This is the original stack, less the top j states.

2. *Terminal insertion*. For every terminal T that is accepted by state S_j , insert T before X in the input string and attempt to parse the input string through token X (but not past X). If a T exists such that the parse is successful, we have found a patch. Otherwise, restore the stack (after each trial) and continue.

3. *Nonterminal insertion*. Let S' be some state that is compatible with S_j . (There exists a transition from S_j to S' on a nonterminal in the complete LR FSA.) Push S' and attempt to parse X with the resulting stack. If an S' exists such that the parse is successful, we have found a patch. Otherwise, restore the stack and continue.

4. *Dropping a state*. If $j = n$, go to step 5. Otherwise, set $j := j - 1$. Attempt to parse through X with the new stack. If successful, we have a patch. Otherwise, restore the stack and go to step 2.

5. *Bottom of stack*. Here $j = n$. Set $j = 0$. Restore the stack. Read the next token, call it X , then go to step 2.

Upon finding a patch in the above algorithm, the stack should be restored and ERFLAG reset. We know that the stack will represent a viable prefix, and that a parse through the current next token X must succeed. Whether the parse will succeed through subsequent tokens is unknown.

The nature of the patch depends on the order in which insertion tokens are produced in steps 2 and 3. It may happen that several alternative patches are possible with a given j and X ; the algorithm simply accepts the first one that it finds.

The nonterminal insertion operation 3 may be eliminated through the use of error productions, as explained in section 12.3.3. Elimination of this step eliminates considerable difficulty with the creation of a suitable semantics stack structure associated with the nonterminal or, more accurately, places their creation in the apply operations where they belong.

Discussion

We have found that the recovery with this simple strategy is not particularly effective. There are several problems as follows:

- There is a tendency to remove more states from the stack than is really necessary. Spurious error reports are often the result of removing too many states from the stack.
- Often an inserted token is an unfortunate choice. For example, we found that a relatively common syntax error caused the token CASE to be inserted under this algorithm. The CASE statement is highly structured, and the insertion of CASE is rarely needed, hence many subsequent spurious errors were generated.
- The information in the input list past the error token X is not used. This information could be used to more effectively select j or an insertion token or to force more input scanning.

12.3.6. Variations on the Bounded-Range Strategy

Several variations on the bounded-range strategy can be devised to correct some of the deficiencies given above. Of these, we have studied error recovery based on the first two rather extensively. A discussion of the strategy's effectiveness is given in section 12.3.9.

1. The tokens that may be inserted may be limited to a subset of the grammar's alphabet. In addition, a limit may be placed on the degree to which the stack can be reduced by dropping states. We find the following guidelines to be effective.

(a) Select a set of tokens that should not be inserted. We feel that these should be terminal tokens and that each one would, if it were to be inserted, demand some subsequent structured form that is unlikely to be present in the source. Candidates for the "don't insert" list are such keywords as PROCEDURE, IF, CASE, WHILE, VAR, to draw from Pascal as an example language.

(b) Select a set of states S' such that if $S' = S_j$, then j will not be decremented. State S' can best be chosen by selecting a set of tokens T' such that the states S' are accessed by the members of T' . We feel that the members of T' should be those terminals and nonterminals that represent the heads of major structures in the language. Such tokens as PROCEDURE, CASE, BEGIN, etc. are reasonable candidates for this list.

Once these two lists have been chosen, the error recovery strategy given above is modified by permitting an insertion only of tokens not in the list (a) and not reducing the stack below a state in the set S' , list (b).

These lists provide a means of tuning the recovery algorithm for a particular language and its grammar. Effectively, they convey information to the recovery algorithm regarding the kinds of errors that are expected in practice. It is our opinion that certain kinds of error are difficult to correct satisfactorily by whatever means. These usually stem from failure to note critical keywords that introduce major program structures. If one of these keywords is misspelled or absent, it is hard to imagine an error recovery strategy that can infer that problem without a major analysis of the remaining input list, especially if some other defective patch is compatible with the information at hand (stack and following few tokens).

Given such a difficulty with critical keywords, it is reasonable to conclude that, if present, they are important recovery indicators, not to be discarded lightly, and if absent, they should not be inserted.

2. As an improvement on variation (1), report a syntax or semantic error only if no error has been reported on the last m tokens, where m is some small number. That is, if an error is seen on a token a_0 in the string $a_0 a_1 \dots a_k \dots$, then we report an error found at token a_k only if $k > m$. Note that only error reports are affected, not the recovery. This variation results in an improvement, however, since the chances are good that an error found in the wake of a previous error is spurious and caused by poor recovery rather than a genuine error. Since each error resets the token count, several sequential error reports may be suppressed.

Of course, this improvement increases the risk that a real error will go unreported.

3. If the compiler is feedback-free, accept a patch with the above error recovery strategy only if the next m tokens in the input list are accepted, where $m > 1$. By attempting to scan more than one token in the input list, the chances of finding a valid recovery are increased. Essentially, there are more clues if more tokens are considered; the possible patches are more constrained. Of course, if no patch can be found, input tokens must be dropped. Eventually, either the source is completely scanned or a patch is found.

This variation may be combined with (1) and (2) above, as an improvement.

12.3.7. Forward Move

Given that the objective of error recovery is to continue parsing with a minimum of subsequent spurious error reports and with a maximum of checking of the subsequent input text, it would appear that the input text past the error point is of considerable importance to an effective error recovery strategy.

We owe the notion of a *forward move* to Graham and Rhodes (Graham

[1975]), who studied error recovery in an operator precedence parser. A forward move is essentially an attempt to reduce, through parsing actions, a portion of the text following the error token before selecting a recovery strategy. Those reductions are made that would have to be made in any parse. A forward move reduces the number of trials needed for a feasible error recovery solution and increases the forward context information needed for an effective recovery.

With operator or simple precedence, a forward move is relatively simple. The error token and the remaining input list is assumed to be some string to be parsed, with the usual "start" token included as a prefix. Reductions are then made as usual, halting when another error is found. Recall that an error is detected through either a token pair that is not in the precedence table, or through a handle that fails to match any production right member. The forward move may also continue to the end of the input list.

Penello [1978] has developed a forward move algorithm suitable for use with an LR(1) parser. In his scheme, an LR parse is begun just past the error token (it could also just as well begin with the error token), with a special parser called an *error parser*. Each state of the error parser represents a set of states of the LR(1) parser. The initial state consists of all those read states that can accept the first input token of the remaining input list. The error parse continues through read, lookahead, and apply actions, just as in an LR(1) parser, except that every action must agree (in a sense to be described more precisely next) and any apply action must be such that there are sufficient states on the stack to support the action.

Let Q' represent a set of states in the error parser and T some token to be next read. Then a transition in the error parser on (Q', T) is permitted only if each of the states q' in Q' either (1) have no move on T or (2) those that show a move agree in action. That is, if any of the Q' states call for a READ action, then every state must either call for READ or for ERROR. (An ERROR simply means that some state will be dropped from the state set on the move.) If any of the Q' states call for a APPLY, then every state must either call for a APPLY on the same production or for ERROR. If any of the Q' states are ACCEPT, the forward move halts. Finally, if an APPLY move is indicated, and there are insufficient states in the forward move stack to support the apply action, the forward move halts.

A special stack is used for the forward move, initially containing only the state set accepting the first token. Given a READ action, a set of states representing the successor states of the members of Q' on token T is pushed on this stack. Given an APPLY action such that sufficient states exist on the stack to support the action, those states are popped and replaced by the indicated apply set.

The action of the forward move is essentially to perform a number of reductions of the input following the error token. It condenses the information in the input list and can be used to devise a patch. It is possible that the

forward move may continue to the end of the program and perform a large number of reductions. It may also halt within one or two tokens, either due to another syntax error, or for some other reason.

Consider a right-most derivation

$$S \rightarrow^* vAy \rightarrow vwxy$$

where $A \rightarrow wx$ is a production. vw is some viable prefix of the sentential form $vwxy$. We say that z is a *viable fragment* of the viable prefix if z is a suffix of vw . Further, we say that U is a *derived viable fragment* of the sentence suffix zy if (1) U derives z , and (2) during a parse of any sentence ending in zy , at some point the parser must reduce z to the viable fragment U .

Penello shows that his forward move yields a viable fragment U that represents the largest possible subsequent input string. Let that string be z . Then $U \rightarrow^+ z$ is a derivation that must be made eventually by any parse.

12.3.8. Correction Strategies with a Forward Move

Pennello proposes a heuristic for error correction, given an existing state stack and a forward move. It is a variation on the bounded-range error correction strategy given in section 12.3.5. However, he can make effective use of the forward move information by looking for a correction that incorporates the states beyond the error point. This amounts to requiring that any repaired configuration be parsable far beyond the error token. An experimental evaluation of his scheme is given in 12.3.9.

Mickunas and Modry (Mickunas [1978]) propose reversing the parse represented by the state stack in some cases in order to achieve an effective patch. This involves much more than just dropping states—it must be possible to reconstruct the parse steps in reverse and obtain some previous stack. A parse reversal can only be achieved by maintaining a complete history of the parse, or by keeping on hand the tokenized input string, along with input string positions in the stack. This information must be kept for any parse, and represents a cost of compiling error-free as well as erroneous source text. Parse reversal also carries significant semantic implications—the semantic actions should also be reversed if the intention is to continue semantic actions through an error recovery.

However, if a parse reversal is possible, their approach apparently yields a good repair of a number of otherwise difficult syntax errors.

Their algorithm consists of two phases—a *condensation phase* and a *correction phase*. The condensation phase is essentially a forward move. A set of states S that can shift on the error token is determined, and a parse is made of the remaining input string, using one of the states in S as a start state. A forward move terminates in one of two ways: (1) an attempt is made to reduce

over the error point, or (2) another error occurs. In case 2, the parse configuration is called a *holding candidate*, and is held for possible use when other strategies fail. In case 1, the configuration is called a *correction candidate*.

At least one correction candidate will always be found—it may be the error configuration itself. If several are found, they are independently examined.

The *correction phase* is a systematic recovery strategy that incorporates the following unit operations:

- Insertion of a terminal token.
- Retreating the parse. By this, they mean reversing the parse represented by the state stack to some former configuration
- Dropping a state representing a terminal token from the stack. If no terminal token is on the stack top, then a parse must be reversed until one is. When a state is dropped, the set of possible insertion tokens changes.
- Assigning a cost to each insertion and each stack deletion, and accumulating a total cost for some strategy path.
- Abandoning a strategy path when its cost exceeds a preset limit and launching an alternative path.
- If all else fails, a holding candidate is selected, and the entire condensation-correction strategy is recursively invoked on it. Recall that a holding candidate is generated when a second parsing error is encountered during a forward move.

There are usually many different paths to follow, depending on the results of the condensation phase and on the number of insertions that are found to be compatible with some stack and the condensation states. A path is followed until either a successful repair is found, or the net cost exceeds a preset threshold. All the paths are followed, and a repair with least cost (if any) is selected.

The correction phase will never drop the error token. If it succeeds in finding a path involving one or more of the operations listed above it reports success, and an effective recovery patch is found. If it fails, then the error token must be dropped and new condensation and correction phases must be launched.

This error recovery scheme was implemented on a small grammar containing about 40 productions and 356 states. Some examples of recovery follow.

Example 1.

The program segment

```
... READ A B[20] WRITE A; GOTO ...
```

should carry a semicolon just before the WRITE. The error configuration is

```
... READ <input-list> <identifier> [ <expr> ]
      ? WRITE A; GOTO ...
```

where “?” marks the error. The condensation phase produces one correction candidate:

```
... READ <input-list> <identifier> [ <expr> ]
      ? <statement> ; GOTO ...
```

associated with two possible reductions:

```
<statement> ::= G = <identifier> : <statement>
and
<statement-list> ::= <statement-list> <statement> ;
```

In the first case, no single insertion repair is possible, so the parse must be reversed. Token “]” is dropped, <expr> is then expanded to “20”, the “20” is dropped, the “[” is dropped, and this path terminates due to excessive cost. (Each insertion or deletion costs 2 units. A path is terminated when the net cost exceeds 5 units.)

This leads to the second case. Here the insertion “;” provides a repair at a cost of 2.

Example 2.

The program segment

```
... X := Y : A := B ; GOTO ...
```

is repaired if the colon after Y is replaced with a semicolon or “:=”. The error configuration is

```
... <statement-list> <leftpart> <identifier> :
      ? A : = B ; GOTO ...
```

In order to see why this configuration is reached, a <leftpart> production is

```
<leftpart> ::= <identifier> : =
```

The characters “:” and “=” are considered separable for parsing purposes, hence a missing “=” component of the expected “:=” has produced the given error configuration.

The condensation phase produces two correction candidates:

```

. . . <statement-list> <leftpart> <identifier> :
      ? <assignment> ; GOTO . . .
and
. . . <statement-list> <leftpart> <identifier> :
      ? <statement> ; GOTO . . .

```

The first of these stems from an attempted reduction

```
<assignment> ::= <leftpart> <assignment>
```

and the second from two possible attempted reductions:

```

<statement> ::= IF <boolean-expr> THEN
              <statement> ELSE <statement>
and
<statement> ::= <identifier> : <statement>

```

In the first of these, only the `<statement>` following the ELSE can possibly be associated with the `<statement>` following the “?” in the correction candidate. Recall that a forward move disregards the state stack preceding the error point, hence has no basis for disregarding the obviously complicated attempted reduction.

We therefore have three cases, one from the first correction candidate and two from the second.

In the first case, the correction phase finds an insertion repairing the error at a cost of 2 units.

In the second case, the correction phase quickly exceeds the threshold cost with stack deletions. It effectively attempts to find an IF-THEN-ELSE statement in the stack and its reversed parsing. Since no such statement exists, this path can only drop states from the stack until the cost threshold is exceeded.

In the third case, the correction phase backs over “:”, backs over `<identifier>`, expands `<leftpart>` to “`<identifier> :=`” then deletes “`=`” (at a cost of 2), to yield the repaired configuration

```

. . . <statement-list> <identifier> : <identifier> :
      <statement> ; GOTO . . .

```

Example 3

The program segment

```
. . . BEGIN X := Y ; Y := Z ; WRITE X Y ; .
```

requires either dropping the BEGIN or inserting “END ;” before the terminating period. The reduction of this segment eventually yields the error configuration

... <statement-list> BEGIN <statement-list>
 <statement> ; ? .

The condensation phase has only the period to work on; however this token appears as part of the production

<program> ::= <declaration-list> <statement-list> .

It therefore selects this reduction, and yields a correction candidate identical to the error configuration. The correction phase then fails to find a token insertion (the grammar requires insertion of “END ;”, not just “END”). It therefore backs up over “;”, <statement>, <statement-list>, and deletes BEGIN, at a cost of 2. (Backups and reverse parses carry no cost.) This yields a repaired configuration

... <statement-list> <statement-list> <statement> .

The repaired configuration is not right-most canonical; the <statement-list> productions are:

<statement-list> ::= <statement-list> ;
 ::= <statement-list> <statement> ;

Discussion

Mickunas’ scheme appears to yield a very effective recovery in a number of difficult error situations. However, it requires not only a forward move, but a reversal of the parse in the stack. In addition, the repaired configuration will not necessarily represent a right-most canonical sentential form. (See example 3 above.) It is also not clear at what point a path should be terminated. If the termination cost is set too high, then the number of cases to examine expands voluminosely; if set too low, the scheme will drop tokens unnecessarily.

As we have seen, a forward move requires a pure syntax—one in which the recognition of every token can be achieved by the scanner alone with additional semantic information such as might be contained in a symbol table. A parse reversal has more serious semantic implications. It may be simplest to just abandon all semantic actions on the first syntax error, since the needed repairs to the semantics stack are so difficult. However, it then becomes necessary to abandon all hope of checking for semantic errors after the recovery is complete.

12.3.9. Empirical Study of Error Recovery

A realistic view of the error recovery problem is that an optimal solution with a reasonable time bound is unlikely to be discovered in the near future. Error recovery is now, and may continue to be, achieved by heuristic methods, with less than optimal effectiveness.

Given this situation, the relative merit of two error recovery algorithms is impossible to assess on theoretical grounds alone, assuming that reasonable means are used to reduce the computational complexity of the methods.

Nevertheless, we would like a reasonably objective means of evaluating an error recovery strategy, for the sake of selecting an error recovery strategy. We propose an empirical measure of error recovery, based on the systematic analysis of a “random” sample of programs, each containing one error.

There are of course a large number of random variables to consider. The language and its grammar have an important influence on error recovery. The kinds of errors that programmers typically make is also of interest. An occasional poor error recovery is excusable if the strategy performs well on the more frequent errors. For example, the use of a semicolon as a statement terminator (i.e., just before an END or an ELSE) is a common error in Algol 60. A misspelling of PROCEDURE is less common, if for no other reason than the infrequent appearance of the keyword PROCEDURE compared to the semicolon. If a strategy recovers well from semicolon errors, but badly from a misspelled PROCEDURE, then it is likely to be acceptable for all practical purposes.

Finally, the error recovery method itself may require some tuning, for example, in a weight function, or in the selection of “don’t insert” tokens, etc.

We propose evaluating error recovery through an experiment with a large number of programs. The programs need not be identical, but must contain exactly one error each, preferably near the beginning of the source. If the error recovery strategy reports exactly one error and recovers within one token, ready to detect more errors, then it has performed satisfactorily. We find instead that in many cases, two or more errors are reported because of a nonoptimal error recovery, or that a large number of tokens are rejected. We may then interpret the relative number of single error reports and the number of rejected tokens as a measure of effectiveness of the error recovery strategy.

We leave unresolved the issues of language, grammar, typical programs, and typical program errors. Research is clearly needed in these areas. What features of a language or grammar cause error recovery to be particularly difficult? Given a language, what errors are most likely to be made by programmers? We have no answers to these questions.

An Error Recovery Experiment

We now describe an experiment in error recovery in some detail, using the bounded-range strategy described in section 12.3.5, with variations 1 and 2.

In variation 2, a syntax error is reported only if no syntax error was found within the last m tokens. Results are obtained with $m=0$ (i.e., every error reported) and $m=3$ (report made only if no report in the last 3 tokens) and compared.

A single program of 37 lines, called the *basis* program, was used for the experiment. Seventy-five copies were made, and a single error was introduced into each copy.

The error-free basis program is given below. It is written in a language similar to Algol 60. This program contains some outer block declarations, a procedure, some control structures, a procedure call, and several assignment statements with expressions.

```

INTEGER I,J,K;
REAL CONSTANT ARRAY RCA:=(15,17,29,30);
REAL ARRAY R(75);
PROCEDURE SAM(X,Y);
  INTEGER X;
  REAL ARRAY Y;
BEGIN
  INTEGER L:=16,M;
  REAL ARRAY RA(18);
  L:=25*M-RA/(I+J);
  X:=INTEGER(Y(3));
  IF X<25E6 THEN
    BEGIN
      L:=25;
      X:=55
    END
  ELSE
    BEGIN
      L:=15;
      X:=50
    END;
  END;
CASE I+15 OF
BEGIN
2: I:=25;
5..9: J:=55;
ELSE: IF I=5 THEN J:=65;
END;
WHILE J>0 DO I:=I+1;
SAM(J,R(3));
I:=J:=25;
SAM(I,R);

```

```

K:=55;
K:=K+(I-J);

```

The errors introduced into the 75 copies are classified as follows. This classification is based on the modification made to the original source to yield the defective source. In most cases, the error symptom could be interpreted in more than one way, however:

- 10 misspelled keywords (BEIGN for BEGIN, IFF for IF, etc.).
- 1 interchange of two keywords.
- 32 token insertions.
- 18 token deletions
- 10 token substitutions
- 2 misspelled user identifiers
- 2 misplaced statements, i.e., a declaration appearing within executable statements.

Here are two examples of error recoveries, one “good” and another “bad.” These examples also illustrate a particular reporting scheme.

Example 1: A “Good” Recovery

```

.
.
.
10.000      0  1  INTEGER L:=16,M;
11.000      0  1  REAL ARRAY RA(18);
12.000      6  1  L:=25*(M-RA/(I+J));

```

ERROR IN LINE 12.000

A ; WAS FOUND, BUT POSSIBILITIES ARE:)

```

STACK: <START> <DECL-LIST> ; <PROCEDURE-HEAD>
      <PROC-FP-HEAD> <BEGIN> <LOCAL-DECLARATIONS>
      <RREF> := <TERM> <MULOP> ( <EXPR>

```

TOKEN) INSERTED

```

13.000      22  1  X:=INTEGER(Y(3));
14.000      30  1  IF X<25E6 THEN
.
.
.

```

Background: From left to right in the first line is the edit line number (10.000), location of the instruction (0), a BEGIN-END level count (1), and the source line (INTEGER L:= 16,M;).

Line 12.000 contains the syntax error—a missing right parenthesis. The error report given is easily generated from the LR(1) tables and the state stack. The error token is the semicolon found at the end of line 12. A number of reductions and lookahead transitions have occurred, such that only a right parenthesis is acceptable to the top-of-stack state, corresponding to the nonterminal <EXPR>.

The stack is listed, for the sake of compiler development. It would be meaningless to the average user and should be suppressed in released versions of the compiler.

The recovery selected is exactly optimal—insertion of a right parenthesis. Note that a terminal insertion with no stack reduction is attempted first; a parse through the insertion and the following token (;) was successful, and a viable solution has therefore been found.

Example 2: A “Poor” Recovery

```

      .
      .
      .
14k200      30  1  IF X<25E6 THEN
15.000      35  1  BEING
16.000      35  1  L:=25;

```

ERROR IN LINE 16.000 OF MODULE SPLTEXT
BEING IS NOT DECLARED.

ERROR IN LINE 16.000 OF MODULE SPLTEXT
A <IDENTIFIER> WAS FOUND, BUT POSSIBILI-
TIES ARE: :=

SEE PAGE 0001, LINE 16.000 FOR PREVIOUS ERROR

STACK: <START> <DECL-LIST> ; <PROCEDURE-HEAD>
<PROC-FP-HEAD> <BEGIN> <LOCAL-DECLARATIONS>
<STLST> ; <IF-HEAD> <RREF>

TOKEN := INSERTED

```

17.000      41  1  X:=55
18.000      0  1  END
19.000      0  0  ELSE

```

ERROR IN LINE 19.000 OF MODULE SPLTEXT

A ELSE WAS FOUND, BUT POSSIBILITIES ARE: ;
 SEE PAGE 0001, LINE 16.000 FOR PREVIOUS ERROR

STACK: <START> <DECL-LIST> ; <PROC-LIST>
 TOKEN ELSE DROPPED

20.000 0 0 BEGIN

TOKEN ; INSERTED

21.000 0 1 L:=15;

ERROR IN LINE 21.000 OF MODULE SPLTEXT
 L IS NOT DECLARED.

SEE PAGE 0001, LINE 19.000 FOR PREVIOUS ERROR

22.000 2 1 X:=50

ERROR IN LINE 22.000 OF MODULE SPLTEXT
 X IS NOT DECLARED.

SEE PAGE 0001, LINE 21.000 FOR PREVIOUS ERROR

23.000 4 1 END;

24.000 4 0 END;

ERROR IN LINE 24.000 OF MODULE SPLTEXT
 A END WAS FOUND, BUT POSSIBILITIES ARE: <IDENTI-
 FIER>

<PROC/SUBR-IDENTIFIER> . . .

SEE PAGE 0001, LINE 22.000 FOR PREVIOUS ERROR

STACK: <START> <DECL-LIST> ; <PROC-
 LIST> <XSTARTX>

<STLST> ;

TOKEN END DROPPED

TOKEN <PROC/SUBR-IDENTIFIER> INSERTED

25.000 5 0 CASE I+15 OF

26.000 10 0 BEGIN

(Note: Parsing is successful from here on)

.
 .
 .

Here, a misspelled BEGIN triggered three syntax error reports and three semantic error reports. The first report is a semantic report that BEING is undeclared (line 15.000). Since BEING is not recognized as a reserved word, the compiler assumed that it was an identifier.

This semantics report should have followed line 15 immediately. How-

ever, no apply action was taken on BEING until a lookahead of one token was made. This lookahead required the next source record, and the source records were printed as read. Thus line 16 was read, some reductions were made, and eventually the semantic error associated with BEING was detected.

This kind of misdirected error message can be avoided by carrying a source token position indicator in the semantics stack. Then an error associated with some token can be accurately connected to a source line and token.

This semantics report is followed immediately by a syntax error report. The stack indicates that an assignment statement, with BEING the left member, was selected as a feasible recovery solution. This solution is valid for the following line 16, "L:=25"; multiple assignments of the form

$$A = B = C = 25$$

are legal in this language. So far, so good.

However, a BEGIN is missing in this program, and its absence is noted in line 19, when an ELSE is found unconnected with the IF-THEN started in line 14. Note that this connection could have been detected by a forward move. The ELSE is dropped, and a semicolon is inserted to repair problems related to the dropped ELSE.

Something else has happened—the absent BEGIN has effectively caused the end of the procedure SAM in line 18. Then the local variables of SAM become undeclared, resulting in two semantic errors (lines 21, 22).

Finally, the extra END in line 24 must be accounted for; the only viable solution is to drop it, leaving a semicolon out of place. A dummy procedure call (a legal statement) is inserted before the semicolon, and the remaining program is thereby accepted without error symptoms.

Summary of Error Recovery Experiment

The performance of our error recovery strategy in the 75 programs is summarized by the bar charts of figure 12.1. Each chart shows the fraction P (ordinate) of programs that exhibited n error reports (abscissa), with $n = 1, 2, 3, 4, 5$, and greater than 5. Charts (a) and (b) illustrate an error recovery in which every error is reported, and charts (c) and (d) illustrate error recovery with variation 2 (error report suppressed if within 3 tokens of a previous error symptom). Charts (a) and (c) are for syntax errors only, and charts (b) and (d) are syntax and semantic errors. Every semantic error was reported, whether near a previous one or not. If nearby semantic errors were suppressed, we would expect some improvement in graph (d).

Graph (c) shows that the bounded-range strategy is quite effective, if only syntax errors are considered. Over 80% of the recoveries were optimal (one error symptom reported), and none of the recoveries reported more than 5 symptoms. If the semantic errors are also considered, the recovery is only 60% effective (graph (d)), and about 6% of the recoveries generated more than 5 symptoms.

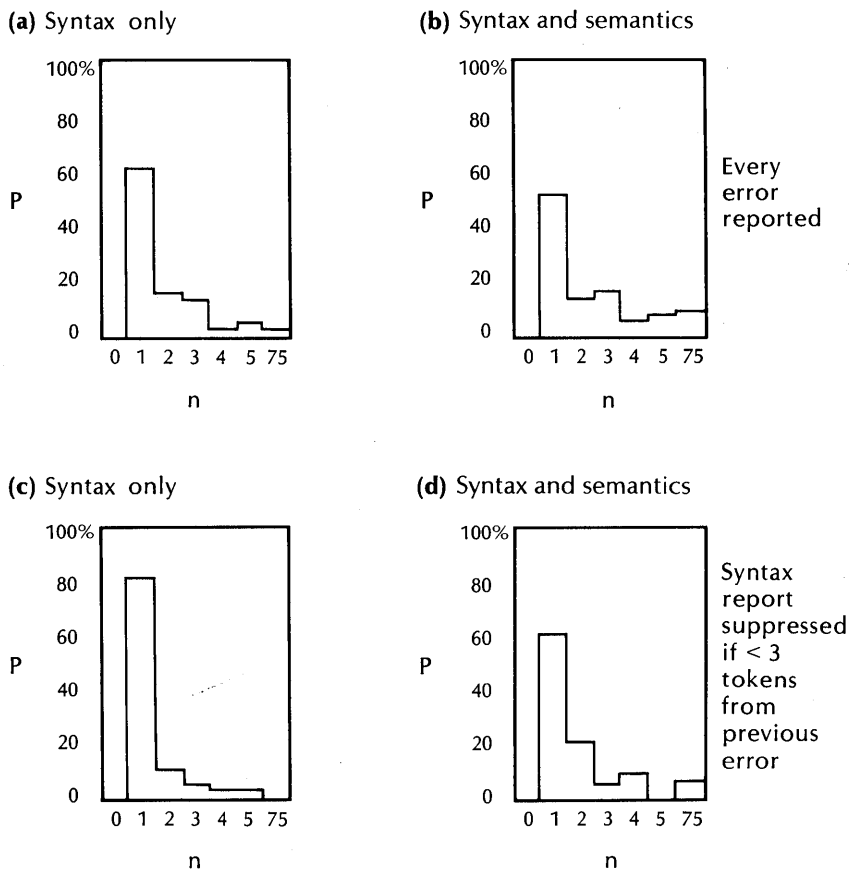


Figure 12.1. Summary of error recovery experiment. P=percentage of programs with n reports, n=number of error reports.

Very few input tokens are dropped in the average recovery, as indicated by the distribution in figure 12.2. No tokens were rejected in about 57% of the cases, and one token was dropped in 27% of the cases. In no case were more than five tokens dropped. Thus 84% of the recoveries were effected by dropping one or no tokens.

These results may be somewhat misleading, since the program chosen was rather small. The recovery is sometimes very bad in a large program. For example, an error in a long list of repeated constants or expressions sometimes causes a cyclic recovery failure in which a large number of spurious errors are reported or a large number of tokens are dropped.

Results of Pennello's Forward-Move Recovery Strategy

Pennello [1978] reports a similar experiment, on 70 Pascal programs. Each

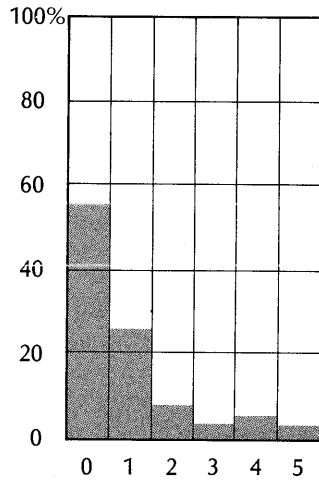


Figure 12.2. Distribution of tokens rejected during error recovery.

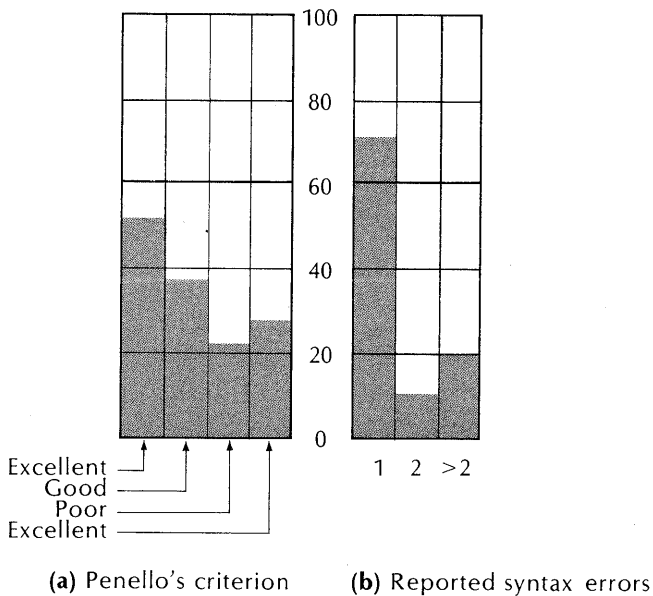


Figure 12.3. Error recovery based on Pennello's forward move algorithm—70 Pascal programs studied.

of these programs is different, but contains one syntax error. They were student programs, with student errors. His results are summarized in figure 12.3. Graph (a) represents Pennello's classification of recovery—"excellent," "good," "poor," and "unrepaired." The "excellent" and "good" categories

are in fact optimal recoveries—in every case the recovery generated just one error report. The “poor” category generated two reports for a single error. The “unrepaired” category are those in which no repair was selected, but the remainder of the program was parsed through the forward move machine rather than the parser. These rob the system of upper-level parsing, and destroy the detection of subsequent errors.

We conclude that Pennello’s system is somewhere between 60 and 90 percent effective, depending on how one regards the “unrepaired” errors. However, these results are for syntax errors only. He does not state how many of the “good” error recoveries would generate spurious semantic error messages.

12.3.10. Recovery in a Recursive Descent Compiler

A recursive descent compiler is closely related to an LL(1) compiler, as we have seen in chapter 4. The stack in a recursive descent compiler is hidden in most implementations; it is the stack used to support recursive procedure calls and local variable allocations. When the stack is so hidden, through the medium of a language implementation, it is impossible to explore stack cutbacks on a trial basis. However, trial parses on the existing stack can be explored through special procedure calls.

Another constraint on error recovery exploration is the distributed nature of the parser. There are no centralized tables that can be consulted in an efficient attempt to find a patch; the parsing system is scattered through many procedures related only by their calls.

Usually, only token deletions and a one-time procedure return is utilized for error recovery. The compiler scans to some characteristic statement end token (i.e., semicolon or END), then forces enough procedure returns to place the system in a state that can accept the string following the statement end. This state is usually some procedure that can accept statements or declarations.

Although crude, this strategy is effective for languages with well-delimited statements such as Fortran, PL/I, or Basic.

Exercises

1. A CASE statement in a certain language requires that every statement in its field of statements carry at least one numeric label. The labels must non conflict. Design an effective semantic error recovery system that will not only report every possible error, but will never cause spurious errors. A sample of such a CASE statement is the following:

```

CASE <expr> OF
  15..25: 28: <stmt> ;
  13: 29: 3: <stmt> ;
  7..8: <stmt>
ENDCASE

```

The pair of periods “..” indicates an inclusive range of labels. Thus “15..17.” is equivalent to “15:16:17.”. Although the labels may be in any order, it should be possible to detect missing labels and conflicts immediately.

2. Carry out an error recovery experiment similar to that given in section 12.3.9 on a commercial Algol, PL/I, or Pascal compiler. What do you conclude?
3. Rewrite the bounded-range recovery algorithm, using variation 3 and no nonterminal insertions. Design suitable data structures for the algorithm, showing that only the usual LR tables are needed, and write the algorithm in a high-level language.
4. Spelling correction is sometimes proposed as an error recovery alternative. Under what circumstances might spelling correction be a useful strategy? What language properties facilitate spelling correction? See Morgan [1970] for a popular spelling corrector.

12.4. Bibliographical Notes

The simplest recovery technique that is essentially language independent is the so-called “panic” solution. When an error is detected, input tokens are discarded until a special terminating symbol, such as “;” or “END” is seen. Then the state stack is erased until a state compatible with this end token is seen. Leinius [1970] reports a slightly more sophisticated version for an LR parser; his is the first error recovery technique reported for an LR parser. James [1972] describes an implementation of Leinius’s method, with some recovery statistics.

In a top-down compiler, the predictive nature of the compiler can be used to insert one or more tokens. Irons [1963] describes such a system.

Levy [1975] and LaFrance [1971] proposed a nondeterministic recovery system that carries out a set of parses, one for each of a set of possibilities. Unfortunately, if this algorithm is carried out for more than a few steps, the resulting computation becomes unreasonably large; as we have seen, such a strategy has exponential complexity.

A minimal error correcting algorithm is given by Aho and Peterson (Aho [1972c]), described earlier.

McGruther [1972] describes a syntax error recovery and correction system that requires the grammar to be both LR and RL. (An RL grammar is such that sentences can be parsed in reverse by an LR parser.) On detecting an error, an RL parser is applied to the remaining string in reverse. The system provides a strong base from which an error analysis may proceed. The essential problem here is to locate particular key tokens past the error point from which a reverse parse may begin; it is impractical to parse from the end of a program back to the error point (that process can halt on another error deeper in the program). The selection of key symbols is discussed at some length; their selection appears to be language dependent.

Graham and Rhodes (Graham [1975]) describe a recovery method for precedence parsers that consists of a forward move followed by a correction step. The forward move performs a sequence of reductions on the input list following the error token; they call this the *condensation phase*. Penello [1978] describes a similar condensation step for an LR parser. Both authors give strategies for the correction step.

Feyock and Lazarus [1976] describe a bottom-up system similar to the bounded-range strategy described above. They propose only insertion, deletion, replacement or interchange of terminal symbols as correction strings. Possible strings are filtered in a number of ways, including semantics checks, to yield a set of viable solutions. If the set is nonempty, one is selected through some language-dependent heuristic tests. Their results are impressive; they claim that thrashing is rarely seen.

Mickunas and Modry (Mickunas [1978]) describe an LR(1) recovery system with a forward move and error region patching. Their approach is described in section 12.3.8.

ANNOTATED BIBLIOGRAPHY

Abbreviations:

ACM: Association for Computing Machinery.

CACM: Communications of the Association for Computing Machinery.

IEEE: Institute for Electrical and Electronic Engineers.

IRE: Institute for Radio Engineers.

JACM: Journal of the Association for Computing Machinery.

SIAM: Society of Industrial and Applied Mathematics

Abrahams [1974], P. W., "Some Remarks on Lookup of Structured Variables," *CACM* **17**, (4) 209–210.

Comment on Gates [1973], on lookup of structured Cobol or PL/I variables, pointing out an error in Gates' approach. Alternative methods are also presented that solve the general problem.

Aho [1969a], A. V., J. D. Ullman, "Properties of Syntax Directed Translations," *J. Computer and System Sciences* **3**, (3) 319–334.

Aho [1969b], A. V., J. D. Ullman, "Syntax Directed Translations and the Pushdown Assembler," *J. Computer and System Sciences* **3**, (1) 37–56.

The pushdown assembler (PA) is a multitape stack machine. Given a syntax-directed translator (SDT) with at most k variables on the right part of any one production, then the pushdown assembler requires $k + 1$ stacks. One stack contains parsing states, and the other k contains strings arising from the syntax-directed transduction. Hierarchy of translators is introduced, and it is shown that the class of PA's is exactly the class of STD's. Rules for constructing the finite control for a given set of productions are given.

Aho [1971], A. V., J. D. Ullman, "Translations on a Context Free Grammar," *Inf. and Control* **19**, 439–475.

Syntax-directed translation, context-free grammars, parse trees, generalized syntax-directed translators, bounds on translation, length relationships, tree automata, conclusions.

Aho [1972a], A. V., J. D. Ullman, *The Theory of Parsing, Translation, and Compiling*, 2 vols., Prentice-Hall, Englewood Cliffs, N.J.

Extensive theoretical framework for languages, grammars, parsers, optimization. (Vol. I) Math preliminaries, introduction to compiling, elements of language theory, theory of translations, general parsing methods, one-pass no backtrack parsing, limited backtrack parsing algorithms. (Vol. II) Techniques for parser optimization, theory of deterministic parsers, translation and code generation, bookkeeping, code optimization. All with formal definitions and many proofs.

Aho [1972b], A. V., J. D. Ullman, "Optimization of Straight Line Programs," *SIAM J. Comput.* **1**, (1) 1–19.

Provides a set of transformations on a sequence of statements into equivalent sequences, then shows that optimization of a straight line sequence under "reasonable" cost criteria can always be accomplished by applying a sequence of these transformations in a prescribed order. Much of this material is also in Aho [1972c].

- Aho [1972c], A. V., T. G. Peterson, "A minimum distance error-correcting parser for context-free languages," *SIAM J. Comput.* **1**, (4) 305–312.
A grammar G for some context-free language is transformed into a grammar G^1 that accepts every sentence in the alphabet of G , by the addition of error productions. A sentence containing one or more errors may be parsed through an Earley parser (see Earley [1970]), and the minimum number of error productions that generates a parse selected. The method operates in $O(n^3)$ steps at worst, where n is the number of tokens in the input sentence.
- Aho [1974a], A. V., S. C. Johnson, "LR Parsing," *Computing Surveys* **6**, (2) 99–124.
Grammars, derivation trees, parsers, parser action and go-to tables, parser construction, parsing ambiguous grammars, parser optimization, error recovery. Very readable, with many examples and illustrations.
- Aho [1974b], A. V., J. E. Hopcroft, J. D. Ullman, *The Design and Analysis of Computer Algorithms*, Addison-Wesley, Reading, Mass.
- Aho [1975], A. V., S. C. Johnson, J. D. Ullman, "Deterministic Parsing of Ambiguous Grammars," *CACM* **18**, (8) 441–452.
Generating an LR parser for an ambiguous expression grammar, then fixing the table to reflect some desired associativity rules for the operators. Results in a considerably smaller table than is otherwise the case.
- Aho [1976], A. V., S. C. Johnson, "Optimal Code Generation for Expression Trees," *JACM* **23**, (3) 488–501.
Algorithms for code generation from expression trees, with theorems.
- Aho [1977a], A. V., S. C. Johnson, J. D. Ullman, "Code Generation for Machines with Multiregister Operations," *Fourth ACM Symposium on Principles of Programming Languages*, 21–28.
Some conclusions regarding register allocation in single and double length results with even assignment for doubles, the problem of optimal allocation.
- Aho [1977b], A. V., S. C. Johnson, J. D. Ullman, "Code Generation for Expressions with Common Subexpressions," *JACM* **24**, (1) 146–160.
Shows that the problem of generating optimal code for expressions containing common subexpressions is computationally difficult, even for simple expressions and simple machines. Some heuristics are given.
- Alexander [1975], W. G., "Static and Dynamic Characteristics of XPL Programs," *Computer*, 41–46.
Statistics on 19 XPL programs—distributions of statements by type, parser reductions, operators in expressions, numeric constants, instructional usage (360 target), branch distances.
- Allen [1976], F. E., J. Cocke, "A Program Data Flow Analysis," *CACM* **19**, (3) 137–147.
Static analysis methods leading to global data flow analysis. Algorithms given that can determine all the definitions which can reach any node of the control flow graph, and all the live definitions.
- Anderson [1973], T., J. Eve, J. J., Horning, "Efficient LR(1) Parsers," *Acta Informatica* **2**, 12–39.
- Aufenkamp [1957], D. D., F. E. Hohn, "Analysis of Sequential Machines," *IRE Trans. EC-6*, 276–285.
- Backus [1957], J. W., *et al.*, "The FORTRAN Automatic Coding System," *Proc.*

- Western Joint Computer Conference* **11**, 188–198.
The original definition of the Fortran language and the first automatic translator of Fortran to IBM 704 machine code.
- Bauer [1968], H., S. Becker, S. L. Graham, "ALGOL W Implementation," Unpublished paper, Stanford University, Stanford, Calif., Computer Science Dept. CS98.
ALGOL W was designed by Wirth, is a simplified Algol 60 language. This paper describes an IBM 360 implementation, is rich in detail.
- Bauer [1974], F. L., "Historical Remarks on Compiler Construction," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 603–621.
Classification of compiler methods and brief historical review of original work and development.
- Beatty [1974], J. C., "Register Assignment Algorithm for Generation of Highly Optimized Object Code," *IBM J. Res. Develop.*, 20–39.
An algorithm that permits a high level of optimization at both local and global levels. Finds appreciable improvement over a conventional production compiler. No attempt to assess implementation manpower costs or expected improvements.
- Berkeley [1964], E. C., D. G. Bobrow (eds.), *The Programming Language LISP: Its Operation and Applications*, The M.I.T. Press, Cambridge, Mass.
A collection of papers on Lisp: The programming system, styles of programming, applications, implementation, many examples of programs. Among the applications: techniques for automatically discovering interesting relations in data; automation of inductive inference on sequences; machine checking of mathematical proofs; an interpreter for string transformations; a language for an incremental compiler.
- Bertsch [1977], E., "The Storage Requirement in Precedence Parsing," *CACM* **20**, (3) 192–194.
A short paper on the compression of precedence tables.
- Bochmann [1976], G. V., "Semantic Evaluation from Left to Right," *CACM* **19**, (2) 55–62.
Describes attribute grammars and their use for the definition of programming languages and compilers. Emphasis on attribute conditions that can be fully evaluated in a single pass over the abstract syntax tree.
- Booth [1967], T. L., *Sequential Machines and Automata Theory*, Wiley, New York.
A general text on finite-state automata. Complete theoretical development, many examples, applications, practical methods and exercises.
- Breuer [1969], M. A., "Generation of Optimal Code for Expressions via Factorization," *CACM* **12**, (6) 333–340.
A complete, but complicated algorithm for finding all factors of a set of expressions to be compiled, then sequencing the operations to minimize the time they need be in memory, then assigning temporary storage cells. Global optimal results are not necessarily obtained.
- Brooker [1963], R. A., D. Morris, "The Compiler-compiler," *Ann. Reviews in Auto. Programming*, **3**, Pergamon, Elmsford, N.Y., 229–275.
The first compiler writing system, using a compressed notation, and recursive descent technique.

- Bruno [1976], J., R. Sethi, "Code Generation for a One-Register Machine," *JACM* **23**, (3) 502–510.
Shows that generating the optimal code for a one-register machine is hard, i.e., in the same class as the traveling salesman problem.
- Brzozowski [1962], J. A., "A Survey of Regular Expressions and Their Applications," *IRE Trans. on Electronic Computers* **11**, (3) 324–335.
One of the early papers on finite-state automata, an exposition of ideas developed by earlier authors. Collects notions of regular expressions, finite-state automata and their transformation.
- Campbell [1976], L., *et al.*, "Draft Proposed ANS FORTRAN," *Sigplan Notices*, **11**, (3).
Modern draft definition of standard Fortran, with syntax graphs, conventions, semantic specifications.
- Chomsky [1956], N., "Three Models for the Description of Language," *IEEE Trans. on Information Theory* **2**, (3) 113–124.
One of the three models in this paper is the phrase-structured grammar. This is the first appearance of this model.
- Cocke [1977], J., K. Kennedy, "An Algorithm for Reduction of Operator Strength," *CACM* **20**, (11) 850–856.
A simple algorithm that uses an indexed temporary table to perform reduction of operator strength in strongly connected program flow regions. The strength of a multiply in a loop can be reduced to an addition, if one of the factors is constant.
- Cohen [1970], D. J., C. C. Gotlieb, "A List Structure Form of Grammars for Syntactic Analysis," *Computing Surveys* **2**, (1) 65–82.
Syntax Graph, graph reductions, compilation from graphs, recursive/nonrecursive compilers, bottom-up parsing with reversed graph.
- Conway [1963], M. E., "Design of a Separable Transition-Diagram Compiler," *CACM* **6**, (7) 396–408.
A Cobol compiler design. Accepts a Cobol subset, operates in small memory, requires two working tapes plus a compiler tape. Use of coroutines as modules.
- DeMorgan [1977], R. M., I. D. Hill, B. A. Wichman, "A Supplement to the ALGOL 60 Revised Report," *Sigplan Notices* **12**, (1) 52–66.
More clean-up on the Algol report.
- DeRemer [1969], F. L., "Practical Translators for LR(k) Languages," Ph.D. thesis, M.I.T., Cambridge, Mass.
Reviews, classifies LR(k) languages and parsers. Introduces SLR grammar (see DeRemer [1971]).
- DeRemer [1971], F. L., "Simple LR(k) Grammars," *CACM* **14**, (7) 453–460.
First paper on SLR(k) grammars. These types are generated by an LR(0) construction process, then use FOLLOW(k) sets to resolve inconsistent states.
- DeRemer [1974a], F. L., "Transformational Grammars," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 121–145.
A case for language processing as a sequence of tree transformations. Lexical, syntactical processing, standardization, flattening, subtree transformational grammars, extension to regular expressions, example of PL/I declaration defactoring.

- DeRemer [1974b], F. L., "Review of Formalisms and Notation," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 37–56.
Concise review of formalisms. Rewriting systems, grammars, Chomsky hierarchy, phrase structures, tree derivation, regular grammars and regular expressions, parsing, parsing strategies, ambiguity, transduction grammars, string-to-tree grammars, self-describing grammars.
- DeRemer [1974c], F. L., "Lexical Analysis," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 109–120.
Lexical terms—scanning, screening, characters, tokens, reserved symbols, regularity. Scanner generation via LR construction, handwritten scanner, error recovery, advisability of including conversion routines.
- Demers [1975], A. J., "Elimination of Single Productions and Merging Nonterminal Symbols of LR(1) Grammars," *Computer Languages* **1**, 105–119.
Formal treatment of single production removal and merging to optimize LR(1) parser tables, with proofs.
- Demers [1977], A., "Generalized Left Corner Parsing," *Fourth ACM Symposium on Principles of Programming Languages*, 170–182.
LC parsing is a combination of LL and LR. Demers develops a parsing machine and demonstrates some minimal properties of the system that facilitate semantic operations through an "announce point" within each production; the announce point is established by the system as near to the left end of the production formula as possible.
- Dijkstra [1976a], E. W. *A Discipline of Programming*, Prentice-Hall, Englewood Cliffs, N.J.
Explores the thesis that a program should derive from a mathematical statement of a problem in a natural way, thereby automatically resulting in a correct program. Introduces a simple but powerful language suited to the task. Semiformal treatment with many interesting examples.
- Dijkstra [1976b], E. W., "On-the-fly garbage collection: an exercise in cooperation," Notes for the 1975 NATO Summer School on Language Hierarchies and Interfaces, in *Lecture Notes in Computer Science*, Springer-Verlag, New York, 46.
- Donovan [1972], J. J., *Systems Programming*, McGraw-Hill computer science series, McGraw-Hill, New York.
An introduction to machine structure, machine language, assembly language, assemblers, macros, loaders, programming languages, compilers, and operating systems. IBM System/360 conventions used extensively. The author was associated with project MAC at M.I.T.
- Earley [1970], J., "An Efficient Context-free Parsing Algorithm," Ph.D. thesis, Carnegie-Mellon University, Pittsburgh, Pa. (1968). Also see *CACM* **13**, (2) 94–102.
The general parser—capable of parsing any sentence in any CFG algorithmically. It runs in linear time for a large class of grammars, is bounded by N^2 for unambiguous grammars, and N^3 otherwise. Shows that it is superior to the top-down and bottom-up algorithms of Griffiths and Petrick by empirical studies.
- Evans [1964], A., Jr., "An ALGOL 60 Compiler," *Ann. Review in Auto. Programming*, **4**, Pergamon, Elmsford, N.Y., 87–124.

- Feldman [1966], J. A., "A Formal Semantics for Computer Languages and Application In a Compiler-Compiler," *CACM* **9**, (1) 3-9.
A system that accepts a production language (Floyd) and a special semantics language keyed to the production language, and yields a compiler. Has symbol table operations and a set of mnemonics to identify kinds of expression/state-ment under consideration.
- Feldman [1968], J., D. Gries, "Translator Writing Systems," *CACM* **11**, (2) 77-113.
Review paper on compiler writing systems. Syntax, syntax trees, grammar, operator precedence, precedence, transition matrices, production language, bounded context grammars, DPDA, LR(k), Tixier's recursive functions of regular expressions. Semantics: TMG (top-down 1-pass c writer), Meta, Cogent, etc. Compiler-compilers: FSL, TGS, CC (Brooker-Morris). Meta-assemblers and extendible compilers: Metaplan, Plasma, Xpop, Algol C, large bibliography.
- Feyock [1976], S., P. Lazarus, "Syntax-Directed Correction of Syntax" *Software-Practice and Experience*, **6**, 207-219.
An error correction method related to XPL bottom-up system, examples, graph of compiler speed vs. error density.
- Fischer [1977], C. N., D. R. Milton, S. B. Quiring, "An Efficient Insertion-Only Error-Corrector for LL(1) Parsers," *Fourth ACM Symposium on Principles of Programming Languages*, 97-103.
Defines a class of insert-correctable LL(1) languages, those for which any error can be corrected by insertion of a suitable terminal string. System will so test a grammar and generate a set of tables that provides optimal string insertion for the correction of syntax errors. Examples.
- Floyd [1961], R., "An Algorithm for Coding Efficient Arithmetic Operations," *CACM* **4**, 42-51.
Specialized method for dealing with arithmetic expression groups to reduce code for single-register machine.
- Floyd [1963], R. W., "Syntactic Analysis and Operator Precedence," *JACM* **10**, (3) 316-333.
Landmark paper on operator precedence. Has an Algol 60 precedence matrix, theory of operator precedence.
- Floyd [1964], R. W., "Bounded Context Syntactic Analysis," *CACM* **7**, (2) 62-67.
A bottom-up parsing method based on context analysis of production right parts.
- Gates [1973], G. W., David A. Poplawski, "A Simple Technique for Structured Variable Lookup," *CACM* **16**, (9) 561-565.
A method for the lookup of structured Cobol or PL/I variables is given. It also checks legality of Cobol identifiers.
- Gill [1962], A., *Introduction to the Theory of Finite State Machines*, McGraw-Hill, New York.
An early text on finite-state automata. Discusses classes of automata, equivalence, reduction and reduction algorithms. Semiformal treatment. (But no treatment of regular expressions or regular grammars).
- Ginsburg [1962], S., *An Introduction to Mathematical Machine Theory*, Addison-Wesley, Reading, Mass.
- Ginsburg [1966a], S., *The Mathematical Theory of Context-Free Languages*, McGraw-Hill, New York.

The first general text on formal context-free language theory. Many general theorems, most original to the author.

- Ginsburg [1966b], S., S. Greibach, "Deterministic Context-Free Languages," *Inf. and Control* **9**, (6) 620–648.
- Glanville [1978], R. S., S. L. Graham, "A New Method for Compiler Code Generation," *Conference Record of the Fifth Annual ACM Symposium on Principles of Programming Languages*, 1978, 231–240.
A construction algorithm for generating machine instructions from a prefix translation of a suitable abstract syntax tree, based on a table representation of the target machine. Conditions for correctness and normal termination of the algorithm are given.
- Graham [1975], S. L., S. P. Rhodes, "Practical Syntactic Error Recovery," *CACM* **18**, (11) 639–650.
Error recovery for precedence parsers, experimental results from several different grammars.
- Gries [1971], D., *Compiler Construction for Digital Computers*, Wiley, New York.
A compiler construction textbook. Emphasis on practical methods: Algol control structures and expressions. Reviews: grammars and languages, scanner, top-down recognizers, simple precedence, other bottom-up recognizers, production language, run-time storage organization, symbol table organization, internal forms of the source, semantic routines introduction, semantics for Algol constructs, storage allocation, error recovery, interpreters, code generation, code optimization, macros, translator writing systems.
- Gries [1977], D., "An exercise in proving parallel programs correct," *CACM* **20**, (12) 921–930.
A parallel program correctness proof method, and its application to an on-the-fly garbage collector (*cf.* Dijkstra [1976], Steele [1975].)
- Griffiths [1974a], M., "Run-time Storage Management," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 195–221.
Classical storage allocation and access. Static allocation, dynamic allocation, block linkage, display, stack compaction, parameter linking, labels/go to, aggregates, lists, garbage collection, storage collapse, parallel processes.
- Griffiths [1974b], M., "LL(1) Grammars and Analyzers," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 57–84.
Predictive Analysis, LL(1) conditions, decision algorithm, recursive descent construction, grammar transformation, semantic insertion, LL(k) grammars, practical results.
- Griffiths [1974c], M., "Introduction to Compiler Compilers," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 356–365.
Brief review of compiler writing systems and current research.
- Hall [1974], A. D., private communication, reported in Waite [1974b].
- Harrison [1965], M. A. *Introduction to Switching and Automata Theory*, McGraw-Hill, New York.
- Harrison [1977], W., "A New Strategy for Code Generation—the General Purpose Optimizing Compiler," *Fourth ACM Symposium on Principles of Programming Languages*, 29–37.

- Uses global flow analysis and an intermediate language of simple primitives to achieve general optimization, regardless of source language form.
- Hartmanis [1966], J., R. E. Stearns, *Algebraic Structure of Sequential Machines*, Prentice-Hall, Englewood Cliffs, N.J.
- Hill [1974], U., "Special Run-time Organization Techniques for Algol 68," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 222–252.
Data storage and management required for Algol 68. Static, dynamic storage, heaps, generative and interpretative handling, special data objects, slicing, rowing, scope checking, scope of procedures, generation of local objects, blocks and procedure calls, actual parameters, garbage collection.
- Hoare [1969], C. A. R., An Axiomatic Basis for Computer Programming, *CACM* **12**, 576–581.
Definition and development of the Hoare axiomatic program correctness proof method, e.g., $A1 \rightarrow A2$; $A1$ and $A2$ are assertions regarding the state of a program system, and S is some executable statement. Proof problem is to show that $A1$ implies $A2$, given execution of S .
- Hoare [1973], C. A. R., N. Wirth, "An Axiomatic Definition of the Programming Language PASCAL," *Acta Informatica* **2**, 335–355.
Brief review of Hoare axiomatic approach, has Pascal statements expressed in axiomatic form, Pascal syntax graphs.
- Hopcroft [1969], J. E., J. D. Ullman, "Formal Languages and Their Relation to Automata," Addison-Wesley, Reading, Mass. Text on formal language and automata theory. Largely superseded by Aho [1972].
- Horning [1974a], J. J., "What the Compiler Should Tell the User," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 525–548.
Normal output, reaction to errors, syntactic errors, other errors, error diagnosis.
- Horning [1974b], J. J., "Structuring Compiler Development," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 498–513.
Goals of compiler development—correctness, availability, generality, adaptability, helpfulness, efficiency. Trade-offs, processes in development: specification, design, implementation, validation, evaluation, maintenance. Management tools: project organization, information distribution and validation, programmer motivation. Technical tools: compiler compilers, standard designs, off-the-shelf components, structured programs, appropriate languages.
- Horning [1974c], J. J., "LR Grammars and Analysers," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 85–108.
Intuitive description, definitions of terms, interpreting LR tables, constructing LR tables, representing LR tables, reduction for efficiency, properties of LR grammars and analysers, some grammar modifications to obtain LR, grammar/language inclusions.
- Huffman [1954], D. A., "The Synthesis of Sequential Switching Networks," *J. Franklin Inst.* **257**, 161–190, 275–303.
- Ingerman [1961], P. Z., "Dynamic Declarations," *CACM* **4**, (42) 59–60.
Algorithm for mapping an OWN array in which dimensions have been

changed—the nontrivial multidimension dynamic array problem is as dealt with here.

- Irons [1961], E. T., "A Syntax Directed Compiler for ALGOL 60," *CACM* **4**, (42) 51–55.
- Irons [1963], E. T., "An Error-Correcting Parse Algorithm," *CACM* **6**, (11) 669–673. A top-down error-correcting parsing system.
- Iverson [1962], K., *A Programming Language*, Wiley, New York.
The original book on APL. Defines the language and gives many examples of its applications to broad areas of computing, e.g., mapping and permutations, ordered trees, graph traversal, microprogramming, matrices, searching, sorting, and the logical calculus.
- James [1972], L. R., *A Syntax Directed Error Recovery Method*, Tech. Report CSRG-13, University of Toronto Press, Toronto.
Table-driven error recovery embedded in an LALR(1) parser. This method drops stack states and input symbols, searches for an insertion or compatible state; uses a 2-symbol lookahead limit then 5-symbol lookahead in dire cases, limits stack cutback through a fixed limit.
- Johnson [1968], W. L., J. H. Porter, S. I. Ackley, D. T. Ross, "Generation of Efficient Lexical Processors Using Finite State Automatic Techniques," *CACM* **11**, (12) 805–813.
Description of the AED RWORD system that accepts regular expressions and generates a finite-state automaton to recognize the expression's language. Has "escape hatches" to provide for unusual lexical constructs. Used in several different compilers as a lexical analyzer.
- Kildall [1973], G. A., "A Unified Approach to Global Program Optimization," *ACM Symposium on Principles of Programming Languages*, 194–206.
- Kleene [1952], S. C., *Introduction to Metamathematics*, Van Nostrand Reinhold, New York.
- Kleene [1956], S. C., "Representation of Events in Nerve Nets," in C. Shannon and J. McCarthy, *Automaton Studies*, Princeton University Press, Princeton, N.J.
- Knuth [1965], D. E., "On the Translation of Languages from Left to Right," *Inf. and Control*, **8**, 607–639.
Original paper on LR(k) languages. Shows that LR(k) languages are equivalent to the deterministic k-symbol lookahead languages, gives two parser construction methods, and proves that the viable prefix set is a regular language.
- Knuth [1968], D. E., *Fundamental Algorithms*, Vol. 1 of *The Art of Computer Programming*, Addison-Wesley, Reading, Mass.
- Knuth [1971], D. E., "An Empirical Study of FORTRAN Programs," *Software—Practice and Experience* **1**, 105–134.
- Kohavi [1971], Z., "Switching and Finite Automata Theory," McGraw-Hill, New York.
- Korenjak [1969], A. J., "A Practical Method for Constructing LR(k) Processors," *CACM* **12**, (11), 613–623.
Large grammar is partitioned into several smaller parts, each of which is parsed independently; the mechanism of parser intercommunication and reporting is developed.

Koster [1974], C. H. A., "Using the CDL Compiler-Compiler," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 366–426.

Detailed review of the CDL compiler writer, a top-down system with symbol table manager system, macros, etc.

Kurki-Suonio [1969], "Notes on Top-Down Languages," *BIT* **9**, 225–238.

A short paper on the properties of LL(k) languages and grammars.

LaFrance [1971], J. E., "Syntax Directed Error Recovery for Compilers," Ph.D. thesis, University of Illinois, Urbana, Computer Science Dept. ILLIAC IV Doc. 249.

LaLonde [1971a], W. R., E. S. Lee, J. J. Horning, "An LALR(k) Parser Generator," *Proc. IFIP Congress* **71**, TA-3, North-Holland Pub. Co., Netherlands, 153–157. See LaLonde [1971b]. Amsterdam.

LaLonde [1971b], W. R., "An Efficient LALR Parser Generator," Tech. Report CSRG-2, University of Toronto Press, Toronto.

Review of LR machines, introduction of LALR(1) parsing algorithm—this finds lookahead sets for inconsistent states based on LR(0) machine and state tracing. However, DeRemer has shown that LaLonde's system actually accepts a subset of the LALR(1) languages.

Lampson [1977], B. W., *et al.*, "Report on the Programming Language Euclid," *Sigplan Notices* **12**, (2).

Euclid draws heavily upon Pascal for its structure and many of its features. The intention is to express programs that are to be verified by formal methods.

Ledgard [1975], H. F., M. Marcotty, "A Genealogy of Control Structures," *CACM* **18**, (11) 629–639.

Review and classification of known control structures, discussion of equivalence and reducibility, hierarchy, examples, four general conclusions.

Lee [1967], J. A. N., *Anatomy of a Compiler*, Reinhold, New York.

Leinius [1970], R. "Error Detection and Recovery for Syntax-Directed Compiler Systems," Ph.D. thesis, University of Wisconsin, Madison.

Mostly simple precedence error recovery treatment, one chapter on LR(k) systems. Uses EULER as model language. Method requires additional states in the LR(k) parser, created heuristically.

Levy [1975], J. P., "Automatic Correction of Syntax-Errors and Programming Languages," *Acta Informatica* **4**, 271–292.

Formal models of error correction. Notion of error, global error correction, local error correction, detailed error correction method for one language construct, problems, practical error correction.

Lewis [1968], P. M., R. E. Stearns, "Syntax-Directed Transduction," *JACM* **15**, (3) 465–488.

Transduction grammars, relation to LR and LL grammars, machines.

Lewis [1971], P. M., D. J. Rosenkrantz, "An ALGOL Compiler Designed Using Automata Theory," *Proc. of the Polytechnic Inst. of Brooklyn, New York, Symposium on Computers and Automata*, New York, 1971. Polytechnic Press of the Polytechnic Institute of Brooklyn; Distributors: Wiley-Interscience, New York. 75–87.

- Loveman [1977], D. B., "Program Improvement by Source-to-Source Transformation," *JACM* 24, (1) 121-145.
User-provided assertions in an Algol program are shown to be valuable in optimization.
- Maley [1963], G. A., J. Earle, *The Logic Design of Transistor Digital Computers*, Prentice-Hall, Englewood Cliffs, N.J.
- Marcotty [1976], M., H. F. Ledgard, G. V. Bochman., "A Sampler of Formal Definitions," *Computing Surveys* 8, (2) 191-276.
Presentation of four well-known formal definition techniques: W-grammars, production systems with an axiomatic approach to semantics, the Vienna definition language, and attribute grammars. Each technique is described tutorially and examples are given; then each is applied to define the same small programming language.
- Maurer [1975], W. D., T. G. Lewis, "Hash Table Methods," *Computing Surveys* 7, (1) 5-19.
Hashing functions, collision, bucket overflow, alternatives to hashing. Limited analysis of efficiency.
- McCarthy [1960], J., "Recursive Functions of Symbolic Expressions and their Computation by Machine," *CACM* 4, (4) 184-195.
The original paper on the mathematical basis of LISP. Five primitive recursive functions are shown to form the basis of a complete programming language. Interpretation of structures as directed graphs.
- McCarthy [1962], J., *et al.*, *LISP 1.5 Programmer's Manual*, The M.I.T. Press, Cambridge, Mass.
A programmer's manual for LISP. Some examples, but best used as a reference document.
- McCluskey [1965a], E. J., *Introduction to the Theory of Switching Circuits*, McGraw-Hill, New York.
- McCluskey [1965b], E. J., T. C. Bartee, *A Survey of Switching Circuit Theory*, McGraw-Hill, New York.
- McCullough [1943], W. S., E. Pitts, "A Logical Calculus of the Ideas Immanent in Nervous Activity," *Bull. of Math. Biophysics* 5, 115-133.
An original paper on finite-state automata and their relation to regular expressions.
- McKeeman [1970], W. M., J. J. Horning, D. B. Wortman, *A Compiler Generator*, Prentice-Hall, Englewood Cliffs, N.J.
A text on compiler construction. Introduction to formalism, LR(k) and precedence parsing. Emphasis on MSP (mixed strategy precedence) method. Contains complete compiler generator programs in XPL and a definition of XPL as a language.
- McKeeman [1974a], W. M., "Compiler Construction," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 1-36.
Broad informal review of compiler components. Definitions, source/target language, implementation language, recursive descent compilers, modularization, specification, feedback-free, vertical/horizontal fragmentation, transformations.

- McKeeman [1974b], W. M., "Symbol Table Access," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 514–524.
Review of linear, sorted, tree, and hash symbol table access methods. Block-structured symbol tables. Contains complete XPL programs and sample traces. Hash functions, secondary stores, evaluation of access methods.
- McKeeman [1974c], W. M., "Programming Language Design," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 514–524.
Who should or should not design languages? Design principles, models for languages: street language, the Algol family.
- McNaughton [1960], R., H. Yamada, "Regular expressions and State Graphs for Automata," *IRE Trans on Elect. Computers* **9**, (1) 39–47. Reprinted in R. Moore, *Sequential Machines: Selected Papers*, Addison-Wesley, Reading, Mass., (1964).
First paper giving algorithms for interconverting state graphs, regular expressions. Deals with all possible regular expressions—union, intersection, complement, closure, concatenation. Theorems and proofs given.
- Mealy [1955], G. H., "Method for Synthesizing Sequential Circuits," *Bell System Tech. J.* **34**, 1054–1079.
- Metcalf [1964], H. H., "A Parameterized Compiler Based on Mechanical Linguistics," *Ann. Reviews in Auto. Programming* **4**, Pergamon Press, 125–165.
A descriptive paper on some recursive descent programming techniques, with special mechanisms for semantic operations. Well suited to a top-down string translator. Informal. Brief discussion of the validation problem (verifying $LL(1)$), but no solution.
- Meyers [1965], W. J., "Optimization of Computer Code", unpublished memorandum, G. E. Research Center, Schenectady, N.Y., 12 pages.
- Mickunas [1976], M. D., R. L. Lancaster, V. B. Schneider, "Transforming LR(k) Grammars to LR(1), SLR(1), and (1,1) Bounded Right-Context Grammars," *JACM* **23**, (3) 511–533.
Grammar transformation results and algorithms.
- Minsky [1967], M., *Computation: Finite and Infinite Machines*, Prentice-Hall, Englewood Cliffs, N.J.
Textbook on finite state automata. Many unusual and interesting examples and side issues.
- Moore [1956], E. F., "Gedanken-Experiments on Sequential Machines," in *Automata Studies*, Princeton University Press, Princeton, N.J., 129–153.
- Morgan [1970], H. L., "Spelling Correction in System Programs," *CACM* **13**, 90–94. A spelling correction algorithm and its applications.
- Morris [1968], R., "Scatter Storage Techniques," *CACM* **7**, (1), 38–43.
- Nakata [1967], Ikuo, "On Compiling Algorithms for Arithmetic Expressions," *CACM* **10**, (8) 492–494.
- Naur [1963], P. (ed.), "Revised Report on the Algorithmic Language ALGOL 60," *CACM* **6**, (1) 1–17.
The original Algol 60 definition.

- Pager [1979], D. "A Practical General Method for Constructing LR(k) Parsers," *Acta Informatica* 7, 249–68.
- Paul [1962], M., "Zur Struktur formaler Sprachen," Universitat Mainz, Dissertation D77.
- Paul [1962a], M., "A General Processor for Certain Formal Languages," *Proc. of the Symposium on Symbolic Languages in Data Processing*, Rome, 1962. Also: Gordon and Breach, New York, 1962, 65–74.
- Pennello [1978], T. J., F. DeRemer, "A Forward Move Algorithm for LR Error Recovery," *Fifth Annual ACM Symposium on Principles of Programming Languages*, Jan. 23, 241–254.
A "forward move" is useful in syntax error recovery. In a forward move, the text past an error token is partially reduced in an attempt to develop information useful in patching over the error. Pennello and DeRemer develop this for an LR parser.
- Perlis [1956], A. J., *et al.*, "Internal translator (IT), a Compiler for the 650," Carnegie Institute of Technology, Computation Center, Pittsburgh. Also: Lincol Lab. Div. 6, Doc. 6D-327.
- Poole [1974], P. C., "Portable and Adaptable Compilers," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 427–497.
Survey of issues of transporting languages and compilers. Problems with current compilers, standards. Survey of techniques: high-level language coding, bootstrapping, language-machine interface, abstract machine modeling. Janus. Case studies: AED, LSD, BCPL, Pascal, IBM Fortran G.
- Purdom [1974], P., "The Size of LALR(1) Parsers," *BIT* 14, 326–337.
Some statistical results enabling prediction of the size of LALR parser tables from some simple grammar measures, based on a large set of grammars.
- Randell [1964], B., L. J. Russell, *ALGOL 60 Implementation*, Academic Press, New York.
Contains a detailed machine model for ALGOL 60 implementation. Review of nested block structure, procedure calls, parameter passing, branching, control structures, etc.
- Rosen [1967], S. (ed.), *Programming Systems and Languages*, McGraw-Hill, New York.
An early collection of papers on languages, compilers, macros.
- Rosen [1973], B. K., "Tree-Manipulating Systems and the Church-Rosser Theorems," *JACM* 20, (1) 160–187.
Considers subtree replacement systems (RPS) and develops sufficient conditions for the Church-Rosser property; indicates that the result of a set of subtree replacements under some replacement system RPS is independent of the order of the replacement. This property does not hold for all tree RPS. Applications to recursive definitions, the lambda calculus and McCarthy's recursive calculus are discussed.
- Rosenkrantz [1970a], D. J., P. M. Lewis, "Deterministic Left-Corner Parsing," *IEEE Conf. Record 11th Annual Symposium on Switching and Automata Theory*, 139–152.
The original left-corner paper. See Demers [1977].
- Rosenkrantz [1970b], D. J., R. E. Stearns, "Properties of Deterministic Top-Down

- Grammars," *Inf. and Control* **17**, 226–256.
 Definitions, Test for LL(k), strong LL(k), e-rules, canonical push-down automata, LL(k) hierarchy, equivalence decidability, properties.
- Sammet [1969], J. E., *Programming Languages: History and Fundamentals*, Prentice-Hall, Englewood Cliffs, N.J.
 A catalog and brief introduction to the reported programming languages known at that time.
- Sammet [1976], J. E., "Roster of Programming Languages for 1974–5," *CACM* **19**, (12) 655–669.
 A sorted list of 167 languages, each described by a dozen lines or so related to availability, implementation, etc. Also see Sammet [1969].
- Sattley [1961], K., "Allocation of Storage for Arrays in ALGOL 60," *CACM* **4**, (42) 60–65.
 General problem of dynamically allocating array space for ALGOL 60.
- Sethi [1970], R., J. D. Ullman, "The Generation of Optimal Code for Arithmetic Expressions," *JACM* **17**, (4) 715–728.
 Given: a machine with $N \geq 1$ general purpose registers, arithmetic instructions that may operate between register/register or register/memory (e.g., IBM 360), and arithmetic expressions. No optimization for common subexpressions. Then two optimal register assignment algorithms are given: one in which no algebraic properties are assumed and another in which certain operators are commutative or both commutative and associative. These algorithms are shown to require a minimal number of storage references in the evaluation and a minimal number of instructions.
- Shamir [1971], E., "Some Inherently Ambiguous Context-Free Languages," *Inf. and Control* **18**, 355–363.
 Introduces a class of inherently ambiguous CFGs.
- Sigplan [1973], "Proceedings of a Symposium on High-Level-Language Computer Architecture," *Sigplan Notices* **8**, (11).
 Nineteen papers on this subject.
- Sigplan [1974], "Proceedings of a Symposium on Very High Level Languages," *Sigplan Notices* **9**, (4).
 Fifteen papers on high-level languages.
- Sigplan [1975a], "Programming Language Design," *Sigplan Notices* **10**, (7).
 Nine short papers on language design. Structured control, data types and program correctness, extensibility, structured languages, abstract data types, exception handling issues, cognitive psychology, and programming language design.
- Sigplan [1975b], "1975 International Conference on Reliable Software," *Sigplan Notices* **10**, (6).
 Sixty-three papers in this area; three are related to languages and their influence on reliable software.
- Sigplan [1976], "Interface Meeting on Programming Systems in the Small Processor Environment," *Sigplan Notices* **11**, (4).
 Twenty-four papers on this subject; four are related to programming languages for small processors.
- Soisalon-Soininen [1977], E., "Elimination of Single Productions from LR Parsers

- in Conjunction with the Use of Default Reductions," *Fourth ACM Symposium on Principles of Programming Languages*, 183–193.
Review of LR construction, development of method for elimination of single production transitions, with concomitant reduction of table size.
- Stearns [1967], R. E., "A Regularity Test for Pushdown Machines," *Inf. and Control* **11**, (3) 323–340.
An algorithm for determining whether a given context-free grammar is regular.
- Stearns [1969], R. E., P. M. Lewis, "Property Grammars and Table Machines," *Inf. and Control* **14**, 524–549.
Formal definition of an attribute system for context free grammars; attributes are effectively attached to derivation tree nodes and used to control bindings and legal derivations.
- Steele [1975], G. L., Jr., "Multiprocessing compactifying garbage collection," *ACM* **18**, (9) 495–508. On-the-fly garbage collector. Also see Dijkstra [1976] and Gries [1977].
- Tanenbaum [1976], A. S., "A Tutorial on ALGOL 68," *Computing Surveys* **8**, (2) 155–190.
Very readable introduction to ALGOL 68, with many examples and good discussion.
- Van Wijngaarden [1969], A. (ed.), "Report on the Algorithmic Language ALGOL 68," *Numerische Mathematik* **14**, 79–218.
Definition of ALGOL 68.
- Waite [1974a], W. M., "Assembly and Linkage," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 333–355.
Model for assembly. Object and statement procedures, cross-referencing, back chaining, storage constraints, two-pass assembly, the RESERVE expression problem, partial assembly and linkage.
- Waite [1974b], W. M., "Optimization," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 549–602.
Classification of techniques: transformations, regions, efficacy. Local optimization: expression rearrangement, redundant code elimination, basic blocks. Global optimization: redundancy and rearrangement, frequency and strength reduction, global analysis.
- Waite [1974c], W. M., "Relationship of Languages to Machines," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 170–194.
Considerations in selecting a suitable interface between a language and a target machine. Data objects: encodings, interpretations, primitive/derived modes, mode conversion, formation rules. Register structure, data access, aggregates, procedures, procedure calls.
- Waite [1974d], W. M., "Semantic Analysis," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 157–169.
Discussion of what to do after abstract syntax tree is formed. Optimizing transformations, attribute propagation, flattening through traversals. Postfix vs. prefix. Operator identification and coercion, semantic ambiguity.
- Waite [1974e], W. M., "Code Generation," in *Lecture Notes in Computer Science*, G. Goos and J. Hartmanis, Springer-Verlag, New York, 302–332.

- A model of code generation, based on Wilcox [1971]. Contains a transducer, simulator. Handles common subexpression optimization.
- Warshall [1962], S., "A Theorem on Boolean Matrices," *JACM* **9**, (1) 11–12. A fast algorithm for computing the transitive closure of a relation, along with a proof. (Also proven in a different way in Aho [1972]).
Prefix- and postfix-translatable objects. Generator data structures, value descriptors, register descriptors, instruction generation, primitive operations, interpretative coding languages. Target of code generation is some assembly language.
- Whitney [1969a], G., "An Extended BNF for Specifying the Syntax of Declarations," *AFIPS Spring Joint Computer Conference*, 801–812.
Formal symbol table functions and grammar extensions, suitable for top-down compiler system for block-structured grammar. Example language (MAL).
- Whitney [1969b], G., "The Generation and Recognition Properties of Table Languages," *Information Processing* **68**, North-Holland, 388–394.
A formal paper on his table language generators, a table automaton, and five closure properties.
- Wirth [1966], N., H. Weber, "EULER: A Generalization of ALGOL, and its Formal Definition," Part 1, *CACM* **9**, (1) 13–25; Part 2, *CACM* **9**, (2) 1966, 89–99. (Part 1) Elementary notation for algorithms, phrase structure grammar, simple precedence, precedence matrix, higher-order precedence. (Part 2) Euler language: precedence matrix and functions, language definition, productions, interpretation of operators, examples.
- Wirth [1971a], N., "The Programming Language PASCAL," *Acta Informatica* **1**, 35–63.
First published definition of Pascal, semiformal.
- Wirth [1971b], N., "The design of a PASCAL compiler," *Software—Practice and Experience* **1**, 309–333.
A description of the design of a PASCAL compiler for the CDC 6000 series, including symbol table structure details, organization of the project, various statistical results on instruction usage, nesting levels, etc. Most of the article is applicable to a Pascal implementation on any machine.
- Wirth [1976a], N., "Programming Languages: What to Demand and How to Assess Them," *Symposium on Software Engineering*, Belfast. What language should do for the user; a strong case for Pascal.
- Wirth [1976b], N., "Professor Cleverbyte's Visit to Heaven," private communication.
A tongue-in-cheek tale of a heaven in which every possible feature of every possible language is implemented in a colossal computer, with such a large operating system that it breaks down 50 times per second (but recovers through elaborate mechanisms).
- Wirth [1976c], N., *Algorithms + Data Structures = Programs*, Prentice-Hall, Englewood Cliffs, N.J.
PASCAL textbook. Applications to data structures, files, sorting, recursive algorithms, dynamic information structures, language structures and compilers. Latter chapter contains a complete top-down definition of grammar and construction of a compiler for a small language (PL/O), including transformation of syntax graphs into program structures, checking for validity, error recovery, scanning, code generation for an Algol-class stack machine.

INDEX

- AOC machine, 377
- APPLY-GOTO table, in LR(1) parser, 230
- AST (abstract syntax tree), 9
 - construction of, 304
- AST evaluation, 316
 - with associative-commutative operators, 571
 - with commutative operators, 570
 - optimal, 562
- AVA, AOC procedure, 422
- Absolute loader, IBM 360, 534
- Acceptance
 - by PDA, 138
 - string, in FSA, 64
- Access methods
 - comparison of, 372
 - see also* String table
- Action function, of (LR(1) parser, 255
- Activation record, in AOC machine, 397
- Active definition, 589
- Acylic graph, 28
- Address constants, IBM 360, 534
- Addressing, IBM 360, 538
- Addressing modes, 473
- Algol renaming rule, 389
- Alignment, data, IBM 360, 528
- Alphabet, 15
- Alteration, 22
 - of regular expressions, 100
- Ambiguity, 43
 - of LL(k) grammar, 149
- Apply, in LR(1) parser, 61
- Apply rule, in extended PDA, 195
- Apply state, in LR(1) parser, 225
- Arithmetic and logical operations,
 - CDC 6000, 517
- Arithmetic expressions, CDC 6000, 521
- Arithmetic, IBM 360, 532
- Arrays, 343
 - in AOC machine, 418
 - dynamic, redimensioning, 429
 - linear mapping of, 345
 - matrix pointer access, 425
 - and matrix pointers, 346
 - multi-dimensional, 343
 - static, 344
- Assemble code, 1
- Assembler, 1
- Assembly language, 1
- Assignment statement, translated to
 - AOC code, 382
- Association, static, 320
- Associativity
 - and commutativity, use of in optimization, 579
 - failure of, 555
- Attribute, 321
 - table, 11, 322
- Automaton, 25
 - deterministic finite-state, 67
 - finite-state, 26, 63
 - linear-bounded, 26
 - non-deterministic, finite-state, 10
 - parsing, 46
 - push-down, 137
 - stack, 137
- Backtracking, 50
 - application to parsing, 51
 - applied to NDFSA, 73
 - limitations of, 53
 - time bound, 56
- Base register, IBM 360, 529, 530, 534
- Base table, IBM 360, 538
- Basic block input/output variables, 603
- Basic reach algorithm, 595
- Block, 323, 575
 - covering of, 323
 - as DAG, 576
 - equivalent, 575
 - in FSA, 64
 - nested, 323
 - normal, 575
 - PDA, 139
 - of statements, 573
 - and storage allocation, AOC machine, 388
- Block entry and exit, in AOC machine, 403
- Block level, 401
- Bounded-range error recovery, 618
 - variations on, 620
- Branch instructions, HP-3000, 492
- Branches
 - CDC 6000, 518
 - IBM 360, 533
- Branches and constants, HP-3000, 503
- CBN (call by name), 438
 - implementation of, 454
 - label parameters, implementation of, 448
 - procedure parameters, implementation of, 446

- CBR (call by reference), 442
- CBV (call by value), 439
 - implementation of, 443
- CDC: *see* Control Data
- CFG: *see* Grammar, context-free
- CST (code segment table), 484
- Canonical derivation: *see*
 - Derivation, canonical
- Closure, of regular expressions, 100
- Code
 - intermediate, 10
 - object, 9
- Code generation, generalized, 543
- Code generator, 9
- Code improvement, 562, 572
- Code segment table, HP-3000, 484
- Comment scanner, 125
- Comments, scanning of, 131
- Commutativity, failure of, 554
- Comparison instructions, IBM 360, 532
- Compilation
 - partial, CDC 6000, 523
 - process, 7
- Compiler, 2
 - cost of, 6
 - multi-pass, 324
 - recursive-descent, 143
 - single-pass, 324
- Concatenation, 17
 - of regular expressions, 100
- Condition code
 - HP 3000, 482, 492
 - IBM 360, 528
- Conditional statement, translated to AOC
 - code, 384
- Configuration
 - of extended of PDA, 194
 - of FSA, 68
 - of PDA, 140
- Conflict, in precedence relations, 199
- Constants, 4
- Control Data 6000, 511
- Control flow graph, 588
- Control structures, 4
- Core items, in LR(0) parser
 - construction, 240
- Counted string, recognition of, 127
- Cross compiler, 2
- Cycle, empty move, of FSA, 76
- DAG (directed acyclic graph)
 - evaluation of, 581
 - reduction of, 580
 - value of, 577
- DFSA: *see* Automaton, deterministic
 - finite-state
- Data flow
 - analysis, 587, 594
 - applications of, 603
 - graph, 597
- Data formats, of HP-3000, 475
- Data item, 588
- Data object
 - access, in Pascal, 434
 - primitive, 334
 - static representations of, 322, 333
- Data structure, in Pascal, 431
- Declaration, 9, 320, 322
- Decoration, of tree node, 31
- Defined variable, 574
- Delimiters, 127
- Derivation
 - canonical, 41
 - left-most, 41
 - left-recursive, 53
 - right-most, 41
- Derivation step: *see* Step, derivation
- Determinism, of FSA, 70
- Dimensions
 - dynamic array, 343
 - static array, 343
- Display, in AOC machine, 378, 411
- Distributivity, failure of, 555
- Dope vector, for array, 418
- Empty cycles, in FSA, 132
- Empty move removal, in FSA, 132
- Empty string, 17
- Equation, regular expression, 111
- Equivalence
 - of FSA, 68, 88
 - LR(1) parser states, 266
- Error
 - correction of, 607
 - diagnosis of, 607
 - effect on compiler, 610
- Error recovery, 11, 607
 - empirical study, 628
 - experiment in, 628
 - recursive descent, 635
- Error report, 607
- Error symptom, 607
- Error token, 609
- Execution, of AOC program, 379
- Expression AST, optimization of, 556
- Expressions, translated to AOC code, 379
- Extended grammar, as FSA, 190
- External symbol dictionary, IBM 360, 536
- FIRST, for extended grammar, 183
- FIRST relation, 213
- FIRST set, 150
- FOLLOW, for extended grammar, 183
- FOLLOW set, 150
- FSA (finite-state automaton)
 - applications of, 122

- program representation of, 120
 - from a regular grammar, 98
 - sparse array representation of, 117
- Feasible state-pair, of pairs table: *see* Pairs table
- Feedback, scanner, 616
- File, intermediate, 10
- Finite-state control, of PDA, 137
- Fixup list, 324
- Flattening, 557
- Fortran, parameter-passing rules, 442
- Forward move, 621
 - correction strategies with, 623
- Free variables, 397, 441
- Function, state transition, 67
- Go, operation in AOC machines, 409
- Goto function, of LR(1) parser, 255
- Grammar, 15, 18
 - arithmetic expression GO, 22
 - augmented, 269
 - augmented, for LR(k) parser, 272
 - classes of, 20
 - context-free, 20, 22
 - context-sensitive, 20
 - extended, 176
 - left-linear, 24
 - LL(k), 148
 - LR(k), 224
 - phase-structure, 20
 - regular, 24
 - right-linear, 20, 24
 - simple precedence, 206
 - uniquely invertible, 206
 - unrestricted, 20
- Grammar transformation, LL(1), 160
- Graph, syntax, 36
- HP-3000, 475
- Halt rule, in extended PDA, 195
- Handle, 194, 195, 199
- Hash access, bounded table, 370
- Hash code, 368
- Hash function, 368, 371
- Header
 - OWN variable correction, HP-3000, 501
 - primary DB, HP-3000, 497
 - procedure call, HP-3000, 499
 - secondary DB initialization, HP-3000, 498
- Heap, in AOC machine, 377
- Hewlett-Packard 3000, 475
- Host language, 2
- IBM System/360, 527
- IL (intermediate language), 465, 466, 470
- Identifier, 321
 - recognition of, 126
- Identifier stack, 361
- Immediate instructions, HP-3000, 490
- Inaccessible entries, of LR(1) parser, 261
- Inadequate state
 - in LR(0) parser, 242
 - resolution of, 246
- Incomplete specification, FSA, 64
- Inconsistent state: *see* Inadequate state
- Index register, IBM 360, 529
- Indirection and indexing, HP-3000, 479
- Input string, of PDA, 137
- Input-output, AOC machine, 388
- Instruction format, CDC 6000, 515
- Instructions
 - HP-3000, 487
 - IBM 360, 529
- Intermediate language, 465, 466, 470
- Interpreter, 1
- Interrupt, program, IBM 360, 528
- Interval, 596
 - higher order, 597
- Interval-based reach algorithm, 600
- Interval head, 596
- Interval ordering, 595
- Isomorphism, of FSA, 69
- Item
 - LR(k), 252
 - in LR(0) parser construction, 235, 236
- k-distinguishability, of FSA, 88
- k-equivalence, of FSA, 88
- Keywords, 126
- Killed definition, 588
- LALR(1) tables, construction of, 268
- LAST relation, 213
- LL(k) grammars, 148
- LR(k) table size, 273
- LR(0) parser construction, 235
 - proofs, 243
- Labels and GOTO's, in AOC machine, 413
- Labels, Algol statement, 329
- Language, 18
 - ambiguous, 46
 - assembly: *see* Assembly language
 - elements, 15
 - of extended PDA, 195
 - of FSA, 68
 - of a grammar G, 27
 - host: *see* Host language
 - object: *see* Object language
 - of PDA, 140
 - source: *see* Source language
 - strongly-typed, 338
- Left-recursion and LL(k) grammar, 149
- Lexical analyzer, 8, 124
- Linker, HP-3000, 494

- Linking, 465
 - CDC 6000, 523
- Linking loader, IBM 360, 534
- Literal conversion, 125
- Literals, 122, 321
- Live definition, 589
- Live information, data flow, 599
- Load instructions, IBM 360, 531
- Load module, IBM 360, 534
- Loader tables, 465
 - structure, 473
- Local variables, HP-3000, 501
- Locally available definition, 588
- Locally exposed use, 589
- Lookahead, in LR(1) parser, 61
- Lookahead state, in LR(1) parser, 225
- Lookahead string, of LR(k) item, 252
- Lookahead table, in LR(1) parser, 230
- Loop code movement, 604

- MAS, AOC procedure, 421
- MMAS, AOC procedure, 427
- MSP: *see* Precedence, mixed strategy
- Machine
 - reduced, 88
 - see also* Automaton
- Machine code, 1
- Macro processor, 125
- Matrix
 - relation as Boolean, 201
 - sum and product, 202
 - see also* Array, multi-dimensional
- Memory address instructions,
 - HP-3000, 490
- Memory organization, HP-3000, 476
- Memory reference instructions,
 - HP-3000, 478
- Memory space, allocation of,
 - HP-3000, 493
- Merging, state, of FSA, 76
- Metasymbols, in regular expressions, 101
- Minimal fixed point, of regular expression
 - equation, 112
- Move
 - empty, of FSA, 70
 - of extended PDA, 194
 - of FSA, 68
 - of PDA, 137
- Multipass compilation, 12
- Multiple declaration, 322

- NDFSA
 - to DFSA reduction, 134
 - transformation to DFSA, 75
 - see also* Automaton, nondeterministic
 - finite-state
- NEW, Pascal function, 435
- Natural order, 28
- Nondeterministic FSA, 70
 - Nondeterministic, to deterministic, of
 - FSA, 82
- Nonterminal, 18
 - inaccessible, 40
 - LL(k), 148
 - useless, 40
- Number, as a token, recognition of, 126

- OWN array, in Algol 60, 429
- Object code, 7
- Object language, 1
- Object module
 - design of, IBM 360, 539
 - IBM 360, 534
- Object program, 1
- Objects, dynamic Pascal, 435
- One-pass compilation, 12
- Operator
 - hierarchy of, 220
 - strength, 220
- Optimization, 9, 551
 - machine-dependent, 553
 - machine-independent, 553
 - tree, 9
- Optimizer, peep-hole, 10

- PCAL, HP-3000, 482
- PCR (procedure copy rule), 393, 438
- PDA
 - configuration of, 140
 - and context-free languages, 143
 - extended, 194
 - extended, and CFG, 195
 - finite control of, 137
 - from a CFG, 143
 - language of, 140
 - matching move, 147
 - move, 137
 - replacement move, 147
 - see also* Automaton, push-down
- PDT (push-down transducer), 286
 - apply move of, 287
 - configuration of, 286
 - definition of, 286
 - halt conditions of, 287
 - matching move of, 287
- PL/I
 - name scanner, 349
 - structure, 343, 347
- PSW: (program status word), 528
- PUSH table, in LR(1) parser, 232
- Pairs table, 94
 - reduction method, 134
- Parameter(s)
 - actual, 438
 - formal, 437
 - Fortran procedure, CDC 600, 520
 - of procedures, 437

- Parser, 9, 25, 46
 - bottom-up, 193
 - canonical LR(1), 254
 - LL(k), 156
 - LL(1), 60
 - LL(1), deterministic, 155
 - LR(k), 193, 252
 - LR(1), 60, 225
 - nondeterministic, 58
 - recursive descent, 161. *See also* Recursive descent parser
 - shift-reduce, 195
 - simple precedence, 199
- Parsing, 1, 15, 25
 - bottom-up, left-to-right, 49
 - deterministic bottom-up, 198
 - left-corner, 47
 - nondeterministic bottom up, 193
 - top-down, left-to-right, 47
- Parsing methods, comparisons, 275
- Partial compilation, HP-3000, 493
- Partition
 - refinement of, 89
 - set, 89
- Pascal
 - CDC 6000 implementation of, 524
 - structures, 352
 - user-defined type, 343
- Passes, compilation, 12
- Peep-hole optimization, 468
- Phrase, simple, 19
- Pointer stack, 361
- Postfix, 469
- Postorder, 28
- Postponement of error checking, LR(1) parser, 262
- Precedence
 - equal, 220
 - higher, 220
 - mixed strategy, 219
 - operator, 219
 - weak, 219
 - see also* Parser, relation
- Precedence pair, 205
- Prefix, 469
- Preorder, 28
- Preserved definition, 588
- Procedure(s)
 - AOC machine, 393
 - external, HP-3000, 484
 - typed, 437
- Procedure calls and exits, HP-3000, 482
- Procedure compilation and USL linkage, HP-3000, 496
- Procedure copy rule, 393, 438
- Procedure label, HP-3000, 482
- Procedure parameter mechanisms, 450
- Production, 19
 - LL(k), 148
- Production rule, 19
- Program, object: *see* Object program
- Program, source: *see* Source program
- Program status word, IBM 360, 528
- Push-down transducer, 286
- Quad, 466
- Quoted string
 - recognition of, 127
 - scanning of, 131
- READ table, in LR(1) parser, 230
- Reached definition, 588
- Read
 - in LR(1) parser, 61
- Read head, of PDA, 137
- Read state, in LR(1) parser, 225
- Recursion and AOC machine, 393
- Recursive descent, error recovery in, 635
- Recursive descent parser, 161
 - construction, 167
 - from extended grammar, 178
 - failure of, 171
 - validation, 171, 174, 188
- Reduction
 - of FSA, 88, 90
 - of LR(1) parser, 228, 260
- Referenced variable, 574
- Reference of identifier, 322
- References, symbolic: *see* Symbolic references
- Refinement
 - proper, 89
 - see also* Partition, refinement of
- Reflexive transition closure, 68
- Reflexive transitive completion, of relation, 202
- Register allocation, IBM 360, 541
- Register assignment, 584
- Register conventions, 473
- Register save and restore, CDC 6000, 522
- Register set operations, CDC 6000, 516
- Registers,
 - and arithmetic, CDC 6000, 513
 - HP-3000, 476
- Regular expressions, 100
 - context-free grammar for, 101
 - correspondence to FSA, 104
 - identities in, 103
- Regular grammar
 - and FSA, 97
 - from an FSA, 99
 - regular expression of, 111
- Rehash algorithm, 370
- Relation, 201
 - equivalence, 88
 - precedence, 199, 206
 - reflexive, 88, 201
 - symmetric, 88, 201

- Relation (*cont'd*)
 - transitive, 88, 201
 - transitive completion of, 202
- Relocation, 466
 - CDC 6000, 523
 - IBM 360, 534
 - and linkage directory, IBM 360, 536
- Replacement system, 19
- Representations
 - of FSA, 114
 - of LR(1) parser, 230
- Resegmentation, HP-3000, 494
- Rule, production: *see* Production rule
- SDTS (syntax-directed translation scheme), 279
 - ambiguity, 285
 - generalized, 296
 - output alphabet of, 280
 - simple, 286
 - simple postfix, 290
 - source element of, 280
 - source grammar of, 280
 - target grammar of, 280
 - translation defined by, 281
 - translation element of, 280
 - translation form, 281
 - tree transformations, 282
- SLALR resolution, 247
- SLR resolution, 251
- STT (segment transfer table), 482
- Save area, IBM 360, 535
- Scanner, 8
 - error, 608
 - feedback, 616
 - for PL/1 names, 349
- Scope
 - of identifiers, 323
 - stack, 361
- Screener: *see* Scanner
- Segment transfer table, HP-3000, 482
- Selectors, in Pascal types, 352
- Selector table, LL(k), 155, 158
- Self-compiling compiler, 2
- Self-resident compiler, 2
- Semantic error, 608, 610, 611
- Semantic operations, error recovery, 618
- Semantics, 15
 - of compiler, 279
- Sentence, 15, 27
 - ambiguous, 43
 - unambiguous, 43
- Sentential form, 26
- Set, empty, 17
- Sets, relation on, 201
- Shift-reduce parser, 195
- Shift rule, in extended PDA, 195
- Single production transitions, removal of,
 - LR(1) parser, 265
- Skyline, 60
- Source file, 7, 124
- Source language, 1
- Source program, 1
- Source records, 8
 - and characters, 125
- Sparse array tables, 114
- Stack
 - in AOC machine, 377
 - of PDA, 137
- Stack and heap allocation, AOC machine, 386
- Stack configuration, HP-3000, 482
- Stack machine, 377
- Stack marker
 - HP-3000, 482
 - in AOC machine, 397
- Stack-ops, HP-3000, 487
- Start state, in LR(1) parser, 61
- State(s),
 - accessible, of FSA, 85
 - halt, in FSA, 64
 - inaccessible, of FSA, 79
 - initial or start, in FSA, 67
 - start, in FSA, 63
- Statements, 9
- Static chain, in AOC machine, 407
- Static scope, 323
- Step, derivation, 19
- Stratification, 160
- String(s), 16
 - empty, 17
 - length of, 17
- String table, 361
 - binary access of, 364
 - hash access of, 368
 - linear access of, 363
 - management of, 11
 - tree access of, 366
- String transducers, limitations of, 300
- String translators
 - and arrays, 302
 - branches and procedures, 303
 - and data typing, 301
- Strong LL(k), 155
- Structure(s)
 - control, 9
 - data, 342
 - extended, 177
 - Pascal static, 352
 - program, 9
 - run-time, 377
- Subexpression, common, identification of, 578
- Subroutine call, IBM 360, 533

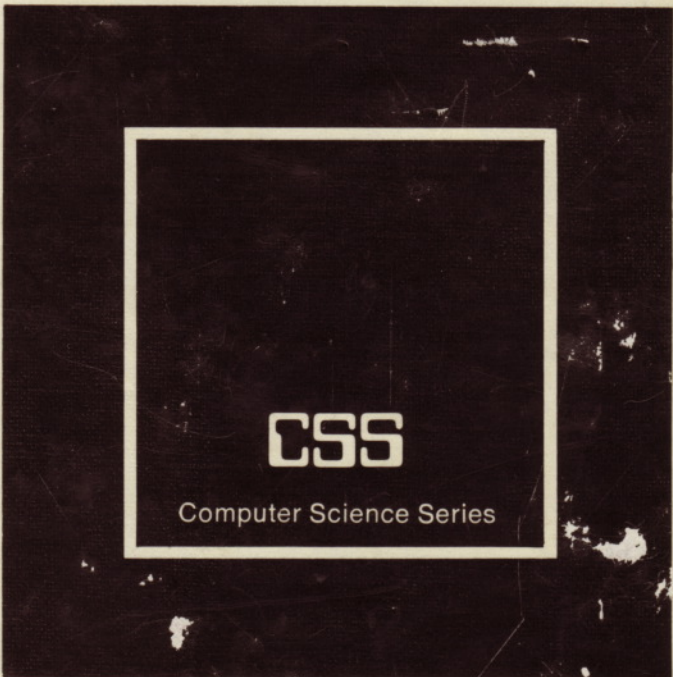
- Subtree, 28
- Symbol, 18
 - start, 20
- Symbol table, 11, 322
 - multi-pass, multi-scope, 327
 - for Pascal, 356
 - single-pass, multi-scope, 325
 - single-scope, 324
- Symbolic locations, 4
- Symbolic references, 2
- Syntactic equivalence: *see* Equivalence
- Syntax-directed translation scheme:
 - see* SDTS
- Syntax error, 608, 609, 613
 - diagnosis of, 614
 - patching of, 615
- Syntax graph: *see* Graph, syntax
- Syntax tree: *see* Tree, syntax
- Synthesis system
 - bottom-up, 309
 - organization of, 312
- Synthesis: *see* Semantics

- TDG (top-down greedy) algorithm, 581
- TFIRST relation, 213
- TPCR: *see* Procedure copy rule
- TRANSPOSE relation, 214
- TRR: *see* Algol renaming rule
- Terminal, 18
- Text table, IBM 360, 536
- Textual address, in AOC machine, 401
- Thunk, 446
- Token assembler, 125
- Tokens, 8, 15
- Transfer point, in AOC machine, 409
- Transitions
 - empty, of FSA, 77
 - empty, of FSA, removal, 79
 - state, in FSA, 64
- Transitive closure, 68
- Translator, 1
 - issues, 11
- Transportable programs, 5
- Tree, 28
 - children, 28
 - complete derivation, 34
 - derivation, 32
 - directed edges, 28
 - frontier of, 33
 - height of, 28
 - immediate ancestor, 28
 - immediate descendants, 28
 - internal node, 28
 - leaf, 28
 - left-to-right natural order, 28
 - level, 28
 - nodes, 28
 - parent node, 28
 - path, 28
 - reduced, 9
 - root node, 28
 - siblings, 28
 - start node, 28
 - syntax, 36
 - terminal node, 28
- Tree dominoes, 34
- Triple, 467
- Type conversion, 339
- Typed procedures
 - implementation of, 443
 - return values of, 445
- Types, 337

- USL: *see* Unsegmented library
- Uniform instruction set, 547
- Uninitialized variables, 603
- Unsegmented library
 - file, structure of, 494
 - HP-3000, 487, 494
- Up-level call, 407
- Upwards exposed use, 589
- Use information, data flow, 599
- Use of identifier, 322
- Used variable, 574
- Useless definitions, 603
- User names, 126

- VPER (value parameter evaluation rule), 439
- Variable, 574
- Vector: *see* Array
- Viable prefix, 205

- WP (working pointer), 413
- Warshall's algorithm, 203
- Wirth-Weber precedence relations, 212
 - direct use of, 215
- With list, Pascal, 359
- With statement, in Pascal, 355
- Working pointer, in AOC machine, 413



CSS

Computer Science Series

REORDER
NUMBER 13-4160